Mathematical software analysis

# Radical Denesting with `Strad.wl`

A unified analysis of three independent reviews,
an experimental evaluation in a live Wolfram kernel,
and a corrected implementation

5 September 2026

---


**Abstract**

`Strad.wl` is a 585-line Wolfram Language program that tries to denest algebraic radicals such as $\sqrt{5+2\sqrt{6}} = \sqrt{2}+\sqrt{3}$ by searching for a multiplier $m$ that makes the minimal polynomial of $(m\,\beta^q)^{1/q}$ share a low-degree factor with the binomial $X^q - m\beta^q$ over an algebraic coefficient field. Three independent reviews of the program were produced earlier; all three were purely static and none executed the code. This document compares the three reviews, reconstructs the algorithm, proves the mathematics that justifies it, and then does what the reviews could not: it runs the program in Wolfram 15.0.1 on more than 400 inputs (classical identities, the reviews' counterexamples, symbolic input, higher-order roots, and randomized families), runs the reviews' own never-executed regression suites, and measures timings.

The experiments confirm the central defect predicted by all three reviews: the program certifies candidates against the principal root of $\beta^q$ instead of against $\beta$, so 39 of 40 random inputs of the form $(p+qi\sqrt{c})^{3/2}$ and 17 of 40 random products of complex square roots come back with the wrong value. The experiments also uncover facts that no review established: the program changes the value of purely numeric input through its custom factorizer; its acceptance test routinely relies on `PossibleZeroQ` answering "unable to decide, assuming zero"; the mistyped prime comparator reverses the prime list, which silently disables the prime-pruning loop and hides an out-of-range slice that only appears once the comparator is fixed; list input yields nonsense; and several "successful" denestings are far uglier than the input. On the positive side, the program denests every classical identity we tried in under a second, and its denominator rationalizer is sound.

A corrected package, `StradFixed.wl`, retains the architecture but certifies every candidate against the original value with exact algebra, makes the factorizer branch-safe, restricts factoring to exact numeric subexpressions, fixes the comparator, slice, cap, and empty-list defects, adds budgets, a visited set, private symbols, a quadratic-surd fast path, candidate polishing and an explicit improvement criterion. On the same battery it produces no wrong value, no kernel message, and finds several denestings the original misses, including the depth-three Borodin–Fagin–Hopcroft–Tompa example $\sqrt{16-2\sqrt{29}+2\sqrt{55-10\sqrt{29}}} = \sqrt{5}+\sqrt{11-2\sqrt{29}}$. The document closes with a prioritized list of future improvements.


**Evidence.** All executions reported here were performed in *15.0.1 for Microsoft Windows (64-bit) (July 2, 2026)* through `wolframscript`. Every table of results is generated mechanically from the recorded runs. Statements about the reviews cite their finding identifiers.

# Contents

# 1 Scope, material, and method

## 1.1 The object under analysis

The file `src/original/Strad.wl` (585 physical lines, SHA-256 `693a2825...cccc93`, byte-identical to the copies shipped with the three reviews) defines a radical denester for the Wolfram Language. Its entry point `Strad[expr]` suppresses diagnostics and calls `DenestRadicals3`, which factors the input, marks every subexpression of the form radicand$^{p/q}$ whose radicand is an explicit algebraic number, groups some products of markers, and hands each marked value to the search engine `denest11`. The engine chooses a root order $q$, forms $\rho = \beta^q$, and searches over multipliers $m$ for a lower-degree factor of $X^q - m\rho$ over an algebraic coefficient field. Helpers implement a minimal-polynomial based denominator rationalizer, nesting-depth annotations, and a custom factorizer. Appendix A reproduces the file with line numbers; all "lines $n$" references point there.

## 1.2 The three reviews

Three independent reviews were written on the same day, 5 September 2026, and are kept under `reports/review-1`, `review-2` and `review-3`. We refer to them as R1, R2 and R3. Table 1 summarizes them. They agree on the architecture, on the most serious defect (the principal-root reset), and on the broad repair strategy. They differ in the defects they add, in their emphasis, and in a few predictions that the live experiments below refine or overturn. Crucially, *none of the three ran the program*: each states explicitly that its Wolfram tests were not executed and that its proposed replacement component is unexecuted.

| | R1 | R2 | R3 |
|---|---|---|---|
| Length | 38 pages, 14 findings F01–F14 | 34 pages, 16 findings B1–B10, H1–H5, M1 | ≈30 pages, 12 findings F1–F12 |
| Executed checks | 45 SymPy/mpmath checks | SymPy/mpmath script | 26 SymPy checks |
| Wolfram execution | none | none | none |
| Proposed code | `VerifiedMultiplierTrial` (one trial) | `CertifiedMultiplierStep` (one trial) | `TryDenestCandidate` + cost policy |
| Regression file | 14 `VerificationTest`s | 15 tests (`.wlt`) | 20 tests (`.wlt`) + runner |
| Distinctive content | complex-product wrapper example; discriminant/index discussion; "what should not be misdiagnosed" | marker `f` and variable `x` collisions; equal-degree gate counterexample; `List @@ Expand` head issue | stale-length slice; quadratic-surd fast path; explicit cost tuple; `combine` unit-entry gap |

Table 1: The three reviews at a glance.

## 1.3 What this document adds

1. A single reconstruction of how the code works, with actual search traces (Section 2), and proofs of the statements that justify or refute its steps (Section 3).
2. A finding-by-finding comparison of the three reviews, including the places where a review over- or under-stated a defect (Section 4).

3. Executed experiments: 90 hand-chosen probes of the original program, 20 helper-level probes, 340 randomized inputs in ten structured families, the reviews' own regression suites, and the same batteries against the corrected version (Section 5).
4. A unified, numbered catalogue of 27 defects and shortcomings, each with its status (confirmed, refined, refuted, or theoretical), its source lines, and the reviews that reported it (Section 6).
5. A corrected implementation, `src/corrected/StradFixed.wl`, that keeps the original architecture and search heuristics but makes correctness independent of them (Section 7, Appendix B).
6. A prioritized list of future improvements (Section 8).

### 1.4 Conventions

Throughout, $\beta$ denotes the exact algebraic number handed to the search engine (the "problem"), $q \geq 2$ the chosen root order, $\rho = \beta^q$ the "radicand", $m$ a multiplier, and $P_m$ the minimal polynomial over $\mathbb{Q}$ of the principal value $(m\rho)^{1/q}$. Rational powers of complex numbers have their Wolfram Language meaning: $z^{p/q} = \exp(\frac{p}{q} \log z)$ with the principal logarithm, so $\operatorname{Arg} z^{1/q} \in (-\pi/q, \pi/q]$. The syntactic radical depth $\operatorname{rdepth}(E)$ is the maximal number of `Power[_, _Rational]` nodes on a root-to-leaf path of the expression tree of $E$; it is a property of the representation, not of the number.

Experiment identifiers such as A02 or D06 refer to the rows of the tables in Section 5. Every result quoted in the text is copied from those mechanically generated tables or from the recorded logs.

## 2 How the code works

### 2.1 Function map

Table 2 lists the twelve top-level definitions and the two local functions. The three layers are an orchestration layer (`Strad`, `DenestRadicals3`), the search engine (`denest11` with its local `trymultiplier` and `newmultipliers`), and helpers.

### 2.2 The wrapper, step by step

Let $E$ be the input. `DenestRadicals3[E, alllevels, func]` proceeds as follows (lines 5–76).

1. **Factor.** $E \mapsto$ `Factorc[E]`. This happens *before* any algebraicity test, on the whole expression, symbolic parts included.
2. **Mark.** Every subexpression matching `radicand_?AlgebraicQ^pow_Rational` is replaced by a marker `f[value, pow]`. With `alllevels =!= True` this is a single `ReplaceAll`, which does not descend into the replaced parts, so inner radicals stay inside the outer marker; with `alllevels === True` it is `ReplaceRepeated`, which keeps marking inside markers and produces nested markers such as `f[Sqrt[55 - 10 f[Sqrt[29]]]]` (observed in the trace of experiment A30). The `value` is a *normalized* radicand: numeric content is split off with `FactorTermsList`, the rest is factored again, Gaussian integer content is decomposed, factors that contain a sum or a complex number with nonzero real part are multiplied together, factors with numerically negative real part (100 digits) are negated with a compensating sign

| Function | Lines | Role |
|---|---|---|
| `Strad` | 1–3 | Unprotects `Print` at load time; calls the wrapper with `Print` dynamically replaced by `Null &`. |
| `DenestRadicals3` | 5–76 | Factors the input, marks $\mathrm{rad}^{p/q}$ subexpressions with a local head `f`, normalizes and groups them, calls the engine, substitutes, simplifies. |
| `denest11` | 79–441 | Chooses $q$, builds initial multipliers, runs the multiplier search with a history of degrees and a work stack. |
| `trymultiplier` | 107–155 | One trial: minimal polynomial, degree gate, polynomial GCD, roots-of-unity orbit, `PossibleZeroQ` filter. |
| `newmultipliers` | 159–211 | New batch of multipliers from the discriminant of a recorded minimal polynomial, or the product of two multipliers. |
| `RationalizeDenominator` | 444–451 | $1/d = -Q(0)/P(0)$ where $P$ is the minimal polynomial of $d$ and $P = (X - d)Q$. |
| `AlgebraicQ` | 454 | `Element[num, Algebraics]`. |
| `nestdepths`, `nestdepthsort`, `maxnestdepth` | 457–498 | Annotate the expression tree with a nesting height for a predicate; group by height; extract the nodes of maximal height. |
| `ComplexitySort` | 501–504 | Sort by `LeafCount`, then by absolute value. |
| `Factorc`, `fullfactor`, `combine` | 507–584 | Custom factorizer: encode products, powers and sums as nested lists, extract common bases from sums, decode. |

Table 2: Function map of `Strad.wl`.

accumulator, each factor is rationalized, and the product is simplified (lines 12–41). This step preserves the value by construction.

3. **Group.** Two markers in one product whose radicands both contain a `Power` are merged into one marker holding the product of the two powers (lines 43–47); markers become one-argument `f[rad^pow]` (lines 48); two one-argument markers whose numerical values have nonzero real and imaginary parts are merged (lines 49–54). All three rules keep the product inside the marker, so they are also value preserving; but they create problems $\beta$ that are products or non-principal powers, which matters in Section 3.2.

4. **Solve.** In single-level mode each distinct marker `f[v]` becomes a rule `f[v] -> func[v]` and the result is `Simplify`d (lines 58–61). In all-level mode a loop repeatedly picks a marker that contains no other marker, solves it, and substitutes the answer into the remaining markers (lines 62–73); this is a hand-written innermost-first resolution.

`Strad[args]` simply wraps this in `Block[{Print = (Null &)}, ...]`. Note what is *not* suppressed: kernel messages. Section 5.2 shows that ordinary successful calls emit `PossibleZeroQ::ztest1` and `N::meprec`.

## 2.3 The engine

`denest11[problem, forcedmultipliers]` (lines 79–441) works on one exact value $\beta$.

1. **Gate.** If $\beta$ contains no `Power` whose base contains a `Power`, return $\beta$ (lines 84–88).
2. **Root order.** $q = \text{lcm}$ of the exponent denominators of the rational-power nodes of maximal nesting height (lines 90–92); $\rho = \beta^q$ (lines 94). A second quantity `extrapower` is computed and printed but never used (lines 95–103).
3. **Initial multipliers.** Unless a list is supplied: expand the rationalized $\rho$, strip rational coefficients from its terms, replace complex atoms by $\pm i$, and for each remaining radical factor $b^{n/d}$ form the complementary factor $b^{(d-n)/d}$; prepend 1 (lines 224–249). For $\rho = 5 + 2\sqrt{6}$ this gives $\{1, \sqrt{6}\}$; for $\rho = \sqrt[3]{2} - 1$ it gives $\{1, 2^{2/3}\}$.
4. **Trial** (`trymultiplier`, lines 107–155). For a multiplier $m$: $P_m = \text{MinPoly}\left((m\rho)^{1/q}\right)$, $d_m = \deg P_m$. If $d_m < d_{\text{best}}$ or $\gcd(d_m, d_{\text{best}}) < d_{\text{best}}$, compute $G_m = \gcd(P_m(X),\, X^q - m\rho)$ with `Extension -> Automatic`. If $0 < \deg G_m < q$: solve $G_m = 0$, divide each root by $m^{1/q}$, multiply by all $q$-th roots of unity, keep the candidates $c$ with `PossibleZeroQ[c - radicand^outerpower]`, take the first, and report a *solution*. Otherwise report *progress* if the degree decreased. If the gate failed, report progress when the degree is different from the best and no worse degree is known yet (lines 147–152).
5. **Search control** (lines 251–432). A `history` of records $\{m, P_m, d_m\}$ is kept sorted by decreasing degree, seeded with $\{0, \infty, \infty\}$. A `multiplierstack` of batches is processed in order; batches are either concrete lists or deferred calls to `newmultipliers`. Progress on a full-set batch schedules a new batch later; progress on a partial batch splits the current batch and inserts the new one immediately. A solution stops the search when it is the first finite-degree record or when the degree dropped by a factor $\geq q$; otherwise the search continues for "additional denesting". After the first pass, products of the promising initial multipliers are formed by `Distribute` and appended (lines 398–428).
6. **New multipliers** (`newmultipliers`, lines 159–211). In the usual branch, partially factor $\text{Disc}(P_{\text{smaller}})$, drop composite cofactors, sort the primes (with the mis-typed comparator, see U5), trim primes separated by a ratio above 100, take a prefix of length $\max(5, \lceil \log 1000 / \log q \rceil)$, and return $m_{\text{smaller}} \cdot \left(\text{Divisors}\left((\prod p_i)^{q-1}\right) \cup \{p_i\}\right) \setminus \{1\}$ sorted by complexity. In the other branch (degrees with a smaller common divisor) return the product of the two multipliers.

## 2.4 Two actual traces

The following is the diagnostic output of `DenestRadicals3[Sqrt[5 + 2 Sqrt[6]]]` in the kernel (Section 5.1). The first two multipliers give degree 4 without a proper GCD; the discriminant $\text{Disc}(P_1) = 2^{14}3^2$ yields the batch $\{2, 3, 6\}$ and $m = 2$ succeeds.

```
{f[Sqrt[5 + 2*Sqrt[6]]]}
problem  Sqrt[5 + 2*Sqrt[6]]
outer root: 2  extra root: 1
multiplier count: 2  multipliers: {1, Sqrt[6]}
multiplier: 1   minpoly degree: 4
adding new multipliers from: 1
multiplier: Sqrt[6]   minpoly degree: 4
adding new multipliers from: Sqrt[6]
multiplier count: 3  from: {1, 4}
multiplier: 2   minpoly degree: 2
success: {2, 2, {{2, 1}}}
history: {{0, Infinity}, {1, 4}, {2, 2}}
```

```
Sqrt[2] + Sqrt[3]
```

For Ramanujan's $\sqrt[3]{\sqrt[3]{2}-1}$ the engine needs $m = 9$, found as the fifth element of the discriminant batch:

```
problem  (-1 + 2^(1/3))^(1/3)
outer root: 3  extra root: 1
multiplier count: 2  multipliers: {1, 2^(2/3)}
multiplier: 1   minpoly degree: 9
adding new multipliers from: 1
multiplier: 2^(2/3)   minpoly degree: 9
adding new multipliers from: 2^(2/3)
multiplier count: 8  from: {1, 9}
multiplier: 2   minpoly degree: 9
multiplier: 3   minpoly degree: 9
multiplier: 4   minpoly degree: 9
multiplier: 6   minpoly degree: 9
multiplier: 9   minpoly degree: 3
success: {9, 3, {{3, 2}}}
history: {{0, Infinity}, {1, 9}, {9, 3}}
(1 - 2^(1/3) + 2^(2/3))/3^(2/3)
```

Indeed $(1-\sqrt[3]{2}+\sqrt[3]{4})^3 = 9(\sqrt[3]{2}-1)$, so the multiplier 9 turns the degree-9 number $(\sqrt[3]{2}-1)^{1/3}$ into the degree-3 number $1-\sqrt[3]{2}+\sqrt[3]{4}$, whose minimal polynomial has a linear common factor with $X^3 - 9(\sqrt[3]{2}-1)$ over $\mathbb{Q}(\sqrt[3]{2})$.

### 2.5 The helpers

**Rationalizer.** `RationalizeDenominator` (lines 444–451) is mathematically sound (Proposition 3.4) but uses the symbol `x` both as the polynomial variable and, through `/. x -> 0`, as a substitution target on the numerator (U11).

**Nesting helpers.** `nestdepths` returns a nested list $\{E,\ \text{child annotations}\ldots,\ h(E)\}$ where $h(E) = \mathbf{1}_{P(E)} + \max_c h(c)$; `maxnestdepth` prunes everything below the maximum height. With the predicate "is a rational power" this is rdepth. The encoding mixes data lists with structure lists and is the reason list-valued input misbehaves (U15).

**Custom factorizer.** `fullfactor` starts from `FactorList` and repeatedly (at most five passes) rewrites each {factor, exponent} pair: a power $\{a^b, c\}$ becomes $\{a, bc\}$; a composite rational becomes its prime factorization; a sum is factored term by term by `combine`, which pulls bases common to all terms out of the sum with their minimal exponent, later multiplied by the outer exponent. `Factorc` decodes the nested lists with five shape rules. The two rewrites in italics are exactly the identities $(a^b)^c = a^{bc}$ and $(FS)^p = F^p S^p$, applied without any branch condition (U2).

# 3 Mathematical foundations

This section collects the statements that justify the engine, the statements that show where it goes wrong, and the two identities used by the corrected version. All three reviews prove variants of Propositions 3.1–3.4; the power-identity conditions of Proposition 3.5 and the quadratic-surd formula of Proposition 3.6 are stated in R3 in a weaker form.

## 3.1 Why the GCD step is meaningful

**Proposition 3.1.** *Let $\rho \neq 0$ and $m \neq 0$ be algebraic numbers, $q \geq 2$, and let $K \supseteq \mathbb{Q}$ be the field generated by the coefficients used by* `PolynomialGCD` *with* `Extension -> Automatic` *(it contains $m\rho$). Let $P_m$ be the minimal polynomial over $\mathbb{Q}$ of $\alpha = (m\rho)^{1/q}$ and $G_m = \gcd_{K[X]}(P_m, X^q - m\rho)$. Then $1 \leq \deg G_m \leq q$, and every root of $G_m$ is a $q$-th root of $m\rho$.*

*Proof.* $\alpha$ is a common root, so the minimal polynomial of $\alpha$ over $K$ divides both polynomials and hence their GCD; thus $\deg G_m \geq 1$. $G_m$ divides the degree-$q$ binomial, which gives the upper bound and the root property. □

A GCD of degree $e < q$ is an equation of degree $e$ over $K$ for some $q$-th root of $m\rho$. In the best case $e = 1$ and that root is an element of $K$ written without the outer radical. Nothing in the proposition says that solving $G_m$ yields a *simpler radical expression*: the roots may involve `Root` objects or nested radicals of their own (experiments Q01–Q08 show candidates containing $\sqrt{9 - 6\sqrt{2}}$, which is itself denestable).

## 3.2 Why the roots-of-unity orbit is right, and why the filter is wrong

**Proposition 3.2.** *Let $\beta \neq 0$, $q \geq 2$, $\rho = \beta^q$, $m \neq 0$, and let $r$ be any root of $G_m$. Then $\{\, r\, m^{-1/q} \zeta_q^k : 0 \leq k < q \,\}$, $\zeta_q = e^{2\pi i/q}$, is exactly the set of $q$-th roots of $\rho$; in particular it contains $\beta$.*

*Proof.* $r^q = m\rho$ and $(m^{1/q})^q = m$ for any choice of $m^{1/q}$, so $(r/m^{1/q})^q = \rho$. The $q$ numbers are distinct since $\rho \neq 0$, and $X^q - \rho$ has exactly $q$ roots. □

No identity of the form $(m\rho)^{1/q} = m^{1/q}\rho^{1/q}$ is used. The code builds precisely this orbit (lines 121–124); the orbit is a strength of the program. The defect is the selection:

```
Select[possibilities, PossibleZeroQ[# - radicand^outerpower] &]
```

compares against $\rho^{1/q} = (\beta^q)^{1/q}$, not against $\beta$.

**Proposition 3.3.** *For $\beta = re^{i\theta} \neq 0$ with $\theta = \operatorname{Arg}\beta \in (-\pi, \pi]$ and an integer $q \geq 2$,*

$$(\beta^q)^{1/q} = \beta\, e^{-2\pi i k/q}, \qquad k \in \mathbb{Z} \text{ with } q\theta - 2\pi k \in (-\pi, \pi].$$

*Hence $(\beta^q)^{1/q} = \beta$ if and only if $-\pi/q < \operatorname{Arg}\beta \leq \pi/q$.*

*Proof.* $\operatorname{Arg}(\beta^q) = q\theta - 2\pi k$ by definition of the principal argument; take the principal $q$-th root. □

Every input to the engine that is itself a principal root ($\beta = \rho^{1/q}$ literally) lies in the sector, which is why the classical examples work. The wrapper, however, sends products of markers and powers $\mathrm{rad}^{p/q}$ with $p > 1$ to the engine; those need not lie in the sector. Example: $\beta = (-1 + 2i\sqrt{2})^{3/2} = (1 + i\sqrt{2})^3 = -5 + i\sqrt{2}$ has $\operatorname{Arg}\beta > \pi/2$, so $(\beta^2)^{1/2} = 5 - i\sqrt{2} = -\beta$, and the program returns $5 - i\sqrt{2}$ (experiment C02). The randomized family F5 shows that this is the typical, not the exceptional, outcome for such inputs.

### 3.3 The rationalizer is sound

**Proposition 3.4.** *Let $d \neq 0$ be algebraic with minimal polynomial $P \in \mathbb{Q}[X]$ (any nonzero rational multiple, not necessarily monic), and let $P(X) = (X - d)Q(X)$. Then $P(0) \neq 0$ and $1/d = -Q(0)/P(0)$.*

*Proof.* $P$ is irreducible of degree $\geq 1$ with root $d \neq 0$, so $P(0) \neq 0$. Evaluating $P(X) = (X-d)Q(X)$ at 0 gives $P(0) = -dQ(0)$. □

This is exactly lines 447–450. Experiments R01–R09 confirm the helper on quadratic, quartic, complex, cubic, and nested denominators.

### 3.4 When power identities are valid

**Proposition 3.5.** *For nonzero complex $a, F, S$ and real exponents:*

1. *$(a^b)^c = a^{bc}$ holds whenever $c \in \mathbb{Z}$, or $b \in \mathbb{R}$ with $-1 < b \leq 1$.*
2. *Let $p = n/q$ in lowest terms with $q \geq 2$. Then $(FS)^p = F^p S^p$ if and only if $\operatorname{Arg}F + \operatorname{Arg}S \in (-\pi, \pi]$. In particular it holds whenever $F > 0$ or $S > 0$.*

*Proof.* (1) If $c \in \mathbb{Z}$, $(a^b)^c = \exp(c\log(a^b))$ and $\exp(c \cdot 2\pi i k) = 1$ absorbs the branch. If $-1 < b \leq 1$ then $\operatorname{Im}(b\log a) = b\operatorname{Arg}a \in (-\pi, \pi]$, so $\log(a^b) = b\log a$ exactly and $(a^b)^c = \exp(cb\log a) = a^{bc}$. (2) $\log(FS) = \log F + \log S - 2\pi i k$ with $k \in \{-1, 0, 1\}$ chosen so that the imaginary part lies in $(-\pi, \pi]$; $k = 0$ iff $\operatorname{Arg}F + \operatorname{Arg}S \in (-\pi, \pi]$. Then $(FS)^p = F^p S^p \exp(-2\pi i k p)$, and $\exp(-2\pi i k n/q) = 1$ iff $q \mid kn$, i.e. iff $k = 0$ since $\gcd(n, q) = 1$ and $|k| \leq 1 < q$. □

The custom factorizer applies (1) with rational $c$ and unrestricted $b$ (lines 527–531) and (2) with rational $p$ and unrestricted $F$ (lines 571–583). Counterexamples for (1): $\sqrt{z^2} \mapsto z$ (E01). For (2): $\sqrt{-x-y} \mapsto i\sqrt{x+y}$ (E03), and, entirely numerically, $F = -1$, $S = \sqrt{2} + (-1)^{1/3}$ with $\operatorname{Arg}F + \operatorname{Arg}S = \pi + \pi/9 > \pi$, where the program returns $i\sqrt{S} = -\sqrt{-S}$ (J01). The corrected factorizer applies (1) only under the stated condition and (2) only when $p \in \mathbb{Z}$, $F > 0$ or $S > 0$.

### 3.5 Quadratic surds

**Proposition 3.6.** *Let $a, b, c \in \mathbb{Q}$ with $a > 0$, $b \neq 0$, $c > 0$ not a square, and suppose $\Delta = a^2 - b^2 c = \delta^2$ with $\delta \in \mathbb{Q}_{\geq 0}$. Put $u = (a + \delta)/2$ and $v = (a - \delta)/2$. Then $u \geq v \geq 0$ and*

$$\sqrt{a + b\sqrt{c}} = \sqrt{u} + \operatorname{sgn}(b)\sqrt{v},$$

*where both sides are principal square roots. If $a < 0$ and $a + b\sqrt{c} < 0$ then $\sqrt{a + b\sqrt{c}} = i\sqrt{-a - b\sqrt{c}}$ and the formula applies to $-a, -b$.*

*Proof.* $a \geq \delta$ because $a^2 = \Delta + b^2 c \geq \Delta$, so $v \geq 0$. The square of the right side is $u+v+2\operatorname{sgn}(b)\sqrt{uv} = a + \operatorname{sgn}(b)\sqrt{a^2-\Delta} = a + \operatorname{sgn}(b)|b|\sqrt{c} = a + b\sqrt{c}$, and the right side is $\geq \sqrt{u} - \sqrt{v} \geq 0$, so it is the principal root. The negative case is $\sqrt{-w} = i\sqrt{w}$ for $w > 0$. □

This is the classical criterion (one half of Landau's theorem for quadratic surds); the corrected version uses it both as a fast path and to clean inner surds that appear inside candidates, e.g. $\sqrt{9 - 6\sqrt{2}} = \sqrt{6} - \sqrt{3}$.

### 3.6 How many multipliers a batch contains

**Proposition 3.7.** *If $t$ distinct primes are selected, the batch built from* Divisors $\left(\left(\prod_{i \leq t} p_i\right)^{q-1}\right)$ *contains exactly $q^t$ elements before the unit is dropped.*

*Proof.* Each divisor is $\prod p_i^{e_i}$ with independent $0 \leq e_i \leq q-1$. □

The code selects $t = \min(\#\text{primes}, \max(5, \lceil \log 1000 / \log q \rceil))$ (lines 190–194), so the nominal cap of 1000 is not a cap: $t = 10$ for $q = 2$ gives 1023 candidates, and the forced minimum $t = 5$ gives $q^5 - 1$ for large $q$ (99 999 for $q = 10$; experiment N06 materializes exactly that list). The corrected version chooses the largest $t$ with $q^t - 1 \leq$ cap in integer arithmetic.

## 4 The three reviews compared

### 4.1 Coverage matrix

Table 3 maps every finding of every review onto the unified catalogue of Section 6. A dot means the review reports the item as a finding; a circle means it is mentioned in passing. The last column gives the experimental status established in Section 5.

### 4.2 Where the reviews agree

All three reviews reconstruct the same algorithm, prove the same three propositions (nonconstant GCD, orbit coverage, principal-root reset), single out U1 and U2 as the critical defects, recognize the rationalizer as sound, and recommend the same repair order: certify against the original value, remove or guard unsafe rewrites, validate inputs and intermediate results, then tune heuristics. Each ships a one-multiplier "certified trial" component with the same shape (validate, minimal polynomial, GCD, orbit, exact certification, structured result). The experiments vindicate this consensus: every wrong value observed is a consequence of U1 or U2, and the corrected version's certification layer eliminates all of them.

### 4.3 Where they differ

- R2 is the only review that notices the two symbol collisions (U10, U11) and the equal-degree gate (U12); all three predictions are confirmed exactly (M01, R05, D06).
- R3 is the only review that notices the stale-length slice (U6) and proposes the quadratic-surd fast path and an explicit cost tuple; the corrected version adopts both. Its F9 remark about `MemberQ[..., Infinity]` excluding level 0 is correct as a reading of the code but moot for

exact numeric input, because the kernel auto-simplifies $(a^{1/2})^{1/3}$ and similar nested powers before they reach the gate.

- R1 gives the most careful "what should not be misdiagnosed" list (sign flips with a compensating accumulator, non-strict comparators, one-argument `MinimalPolynomial`) and a discussion of the discriminant/index relation $\mathrm{Disc}(P) = \mathrm{Disc}(K)[\mathcal{O}_K : \mathbb{Z}[\alpha]]^2$. Its proposed regression `Factorc[Sqrt[t + t^2]]` does *not* exhibit the predicted failure: the kernel returns `Sqrt[t (1 + t)]`, which is value preserving (H02). The mechanism it describes is real, but the concrete probe was mis-chosen; `Sqrt[-x - y]` (R3) and the numeric `Sqrt[-(Sqrt[2] + (-1)^(1/3))]` (R1's own algebraic example, never run) are the probes that fail.
- R2 and R3 correctly hedge that the wrapper keeps a leading minus sign outside the marker, so `Strad[-Sqrt[3 + 2 Sqrt[2]]]` is *not* a wrapper-level counterexample (C01 confirms), whereas the direct engine call is (D01).
- Only R3 notes that `history[[2, -1]]` in the stopping rule is the largest finite degree seen so far rather than a fixed baseline.

### 4.4 What the reviews could not know

Because none of them executed the program, several important facts were invisible to all three:

- The mis-typed comparator does not merely fail to sort: with Wolfram's merge sort it *reverses* the list (S01, N01). Consequently the ratio-pruning loop (lines 184–189) is dead code, the prefix (lines 190–194) selects the *largest* primes, and the stale-length defect U6 can never fire in the original. Repairing the comparator alone activates U6 (N02).
- The acceptance test is not merely heuristic in principle: in the kernel, `PossibleZeroQ` reports *"Unable to decide whether ... is equal to zero. Assuming it is."* on most successful runs (A03, A05, A07, A08, ...; 14/40 of family F1, 38/40 of F4). The program's successes rest on that "assume zero" fallback.
- The factorizer changes the value of purely numeric input end to end: `Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3))]]` returns the negative of the input (J01), with no message.
- `Strad` on a list returns a `Power`, not a list (K14); on a `Root` object it returns a *nested* radical (B07).
- Loading the file prints a `Syntax::com` warning caused by `Throw[, "solution"]` at lines 320.
- Several "successful" denestings are much larger than the input and contain denestable radicals (D07, L04, A30/A31, Q01–Q08).
- Performance is not the practical problem the reviews feared for inputs of this size: every probe finished in under 2.5 s except a 10th root (20 s), and the worst batch materialized 63 multipliers.

### 4.5 The reviews' own test suites, executed

Each review ships Wolfram tests it did not run. We ran them (Section 5.1). The outcomes are exactly as the reviews predicted where they made predictions: on the original program, every preservation contract for U1, U2, U5, U8, U10, U11 fails and every positive control passes. R2's suite:

4 of 11 original-program tests pass (loading test, worked example, rationalizer, all-level control). The guarded one-trial components of all three reviews, executed for the first time here, behave as designed on the negative and complex targets, reject a zero multiplier and root order one, and return structured failures; details are in Section 5.5. None of them is a full denester, as their authors say.

# 5 Experiments

## 5.1 Setup

All runs used `wolframscript` with *15.0.1 for Microsoft Windows (64-bit) (July 2, 2026)* on the machine holding the repository; the license allows one kernel at a time, so every script ran in a fresh kernel in sequence. A small harness (`runner.wl`, Appendix C) loads the program, evaluates each probe under `TimeConstrained` (120 s unless stated), captures every message that is switched on through `Internal‘HandlerBlock`, and records:

- *Equal*: exact equality, `RootReduce[result - expected] === 0`, between the result and the independently known value of the input (for probes without a known closed form, 40-digit numeric agreement with the input is shown instead); **NO** therefore means a wrong value;
- *Depth*: rdepth before and after;
- wall-clock time and the list of messages.

The original program prints its diagnostics; `Strad` suppresses them but not messages, and both are visible in the logs. Tables are generated from the recorded runs by `export_tables.wl`.

## 5.2 The original program

**Classical identities (Table 4).** The program denests all 26 identities in group A that admit a denesting, including cube, fourth, fifth, sixth and seventh roots, sums and products of nested radicals, complex radicands, and Ramanujan's $\sqrt[3]{\sqrt[3]{2}-1}$, each in well under a second. A13, $\sqrt[3]{\sqrt[3]{5}-\sqrt[3]{4}}$, is correctly left alone: the kernel shows that its minimal polynomial has degree 27, so it does not lie in the degree-9 field $\mathbb{Q}(\sqrt[3]{2},\sqrt[3]{5})$, and the closed form $(\sqrt[3]{2}+\sqrt[3]{20}-\sqrt[3]{25})/3$ that we first expected for it is not an identity (`RootReduce` shows that the cube of that expression is a cubic irrational). Two observations temper the success:

- 19 of the 31 rows carry the message `PossibleZeroQ::ztest1` ("Unable to decide … Assuming it is") together with `N::meprec`. The candidate filter did not *decide* equality; it assumed it.
- A27 $(3+2\sqrt{2})^{1/4} \to \sqrt{1+\sqrt{2}}$ is equal and smaller but not shallower; A30/A31, the depth-three example of Borodin, Fagin, Hopcroft and Tompa, comes back as

$$\frac{\sqrt{319-58\sqrt{29}}-10\sqrt{11-2\sqrt{29}}+\sqrt{5}(3\sqrt{29}-16)}{-5+\sqrt{29}-\sqrt{55-10\sqrt{29}}},$$

  which is equal to the input and one level shallower, but far from the answer $\sqrt{5}+\sqrt{11-2\sqrt{29}}$ (found by the corrected version, Table 16).

Table 4: Original program on classical denesting identities. "Equal" is exact equality (`RootReduce`) with the expected value; where an expected value was not supplied, numeric agreement with the input is shown. Depth is the syntactic radical depth before and after.

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| A01 | `Strad[Sqrt[3 + 2*Sqrt[2]]]` | `1 + Sqrt[2]` | yes | 2→1 | 0.146 |
| A02 | `Strad[Sqrt[5 + 2*Sqrt[6]]]` | `Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.040 |
| A03 | `Strad[Sqrt[2 + Sqrt[3]]]` | `(1 + Sqrt[3])/Sqrt[2]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.072 |
| A04 | `Strad[Sqrt[11 + 6*Sqrt[2]]]` | `3 + Sqrt[2]` | yes | 2→1 | 0.016 |
| A05 | `Strad[Sqrt[3 + Sqrt[5]]]` | `(1 + Sqrt[5])/Sqrt[2]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.030 |
| A06 | `Strad[Sqrt[7 + 2*Sqrt[10]]]` | `Sqrt[2] + Sqrt[5]` | yes | 2→1 | 0.030 |
| A07 | `Strad[Sqrt[6 + 2*Sqrt[2] + 2* Sqrt[3] + 2*Sqrt[6]]]` | `1 + Sqrt[2] + Sqrt[3]` *PossibleZeroQ::ztest1, General::stop* | yes | 2→1 | 0.054 |
| A08 | `Strad[Sqrt[12 + 2*Sqrt[6] + 2* Sqrt[14] + 2*Sqrt[21]]]` | `Sqrt[2] + Sqrt[3] + Sqrt[7]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.096 |
| A09 | `Strad[(Sqrt[5] - 2)^(1/3)]` | `(-1 + Sqrt[5])/2` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.027 |
| A10 | `Strad[(2 + Sqrt[5])^(1/3)]` | `(1 + Sqrt[5])/2` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.019 |
| A11 | `Strad[(10 + 6*Sqrt[3])^(1/3)]` | `1 + Sqrt[3]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.029 |
| A12 | `Strad[(2^(1/3) - 1)^(1/3)]` | `(1 - 2^(1/3) + 2^(2/3))/3^(2/3)` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.052 |
| A13 | `Strad[(5^(1/3) - 4^(1/3))^(1/3)]` | `(-2^(2/3) + 5^(1/3))^(1/3)` | yes | 2→2 | 1.063 |
| A14 | `Strad[((11 + 5*Sqrt[5])/2)^(1/5) ]` | `(1 + Sqrt[5])/2` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.031 |
| A15 | `Strad[Sqrt[1 + 2*2^(1/3) + 2^(2/ 3)]]` | `1 + 2^(1/3)` | yes | 2→1 | 0.004 |
| A16 | `Strad[(17 + 12*Sqrt[2])^(1/4)]` | `1 + Sqrt[2]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.018 |
| A17 | `Strad[(99 + 70*Sqrt[2])^(1/6)]` | `1 + Sqrt[2]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.019 |
| A18 | `Strad[(3 + 2*Sqrt[2])^(3/2)]` | `7 + 5*Sqrt[2]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.026 |
| A19 | `Strad[(3 + 2*Sqrt[2])^(-2^(-1))]` | `-1 + Sqrt[2]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.018 |
| A20 | `Strad[1/Sqrt[3 + 2*Sqrt[2]]]` | `-1 + Sqrt[2]` | yes | 2→1 | 0.009 |
| A21 | `Strad[Sqrt[3 + 2*Sqrt[2]] + Sqrt[5 + 2*Sqrt[6]]]` | `1 + 2*Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.030 |
| A22 | `Strad[Sqrt[3 + 2*Sqrt[2]]*Sqrt[5 + 2*Sqrt[6]]]` | `2 + Sqrt[2] + Sqrt[3] + Sqrt[6]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.046 |
| A23 | `Strad[Sqrt[2 + Sqrt[3]] + Sqrt[2 - Sqrt[3]]]` | `Sqrt[6]` | yes | 2→1 | 0.048 |
| A24 | `Strad[Sqrt[-1 + 2*I*Sqrt[2]]]` | `1 + I*Sqrt[2]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.026 |

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| A25 | `Strad[Sqrt[2 + 2*I*Sqrt[3]]]` | `I + Sqrt[3]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.046 |
| A26 | `Strad[Sqrt[-3 - 2*Sqrt[2]]]` | `I*(1 + Sqrt[2])` | yes | 2→1 | 0.009 |
| A27 | `Strad[(3 + 2*Sqrt[2])^(1/4)]` | `Sqrt[1 + Sqrt[2]]` *PossibleZeroQ::ztest1* | yes | 2→2 | 0.030 |
| A28 | `Strad[Sqrt[3 + 2*Sqrt[3 + 2* Sqrt[2]]], True]` | `Sqrt[5 + 2*Sqrt[2]]` | yes | 3→2 | 0.090 |
| A29 | `Strad[Sqrt[3 + 2*Sqrt[3 + 2* Sqrt[2]]]]` | `Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]` | yes | 3→3 | 0.099 |
| A30 | `Strad[Sqrt[16 - 2*Sqrt[29] + 2* Sqrt[55 - 10*Sqrt[29]]], True]` | `(Sqrt[319 - 58*Sqrt[29]] - 10* Sqrt[11 - 2*Sqrt[29]] + Sqrt[5]* (-16 + 3*Sqrt[29]))/(-5 + Sqrt[ 29] - Sqrt[55 - 10*Sqrt[29]])` *PossibleZeroQ::ztest1* | yes | 3→2 | 0.598 |
| A31 | `Strad[Sqrt[16 - 2*Sqrt[29] + 2* Sqrt[55 - 10*Sqrt[29]]]]` | `(Sqrt[319 - 58*Sqrt[29]] - 10* Sqrt[11 - 2*Sqrt[29]] + Sqrt[5]* (-16 + 3*Sqrt[29]))/(-5 + Sqrt[ 29] - Sqrt[55 - 10*Sqrt[29]])` *PossibleZeroQ::ztest1* | yes | 3→2 | 0.118 |

**Non-denestable and trivial inputs (Table 5).** Values are preserved. B01 shows the final `Simplify` rewriting a non-denestable input into a different, not simpler, form; B07 shows a `Root` object turned into a *nested* radical (U20).

Table 5: Original program on non-denestable and trivial inputs.

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| B01 | `Strad[Sqrt[2 + Sqrt[2]]]` | `2^(1/4)*Sqrt[1 + Sqrt[2]]` | yes | 2→2 | 0.031 |
| B02 | `Strad[Sqrt[1 + Sqrt[2]]]` | `Sqrt[1 + Sqrt[2]]` | yes | 2→2 | 0.022 |
| B03 | `Strad[(1 + Sqrt[2])^(1/3)]` | `(1 + Sqrt[2])^(1/3)` | yes | 2→2 | 0.085 |
| B04 | `Strad[Sqrt[2]]` | `Sqrt[2]` | yes | 1→1 | 0.002 |
| B05 | `Strad[2]` | `2` | yes | 0→0 | 0.000 |
| B06 | `Strad[Sqrt[1 + I]]` | `Sqrt[1 + I]` | yes | 1→1 | 0.002 |
| B07 | `Strad[Root[#1^4 - 10*#1^2 + 1 & , 4]]` | `Sqrt[5 + 2*Sqrt[6]]` | yes | 0→2 | 0.006 |
| B08 | `Strad[Sqrt[Sqrt[2] + Sqrt[3]]]` | `Sqrt[Sqrt[2] + Sqrt[3]]` | yes | 2→2 | 0.074 |
| B09 | `Strad[(2 + 2*I*Sqrt[3])^(1/3)]` | `(2 + (2*I)*Sqrt[3])^(1/3)` | yes | 2→2 | 0.261 |

**Branch counterexamples through the wrapper (Table 6).** C02 and C03 are the wrong values predicted by all three reviews: the program returns $5 - i\sqrt{2}$ for $(-1 + 2i\sqrt{2})^{3/2} = -5 + i\sqrt{2}$, and $(\sqrt{2} - i)(\sqrt{3} - i)$ for $\sqrt{-1 + 2i\sqrt{2}}\sqrt{-2 + 2i\sqrt{3}} = (1 + i\sqrt{2})(1 + i\sqrt{3})$. C01, C04, C07, C08 are right because their values happen to lie in the principal sector of Proposition 3.3 ($\mathrm{Arg} = -\pi/2$ is still inside for $q = 2$).

Table 6: Original program on branch counterexamples (wrapper level). "Equal" compares with the true value of the input.

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| C01 | `Strad[-Sqrt[3 + 2*Sqrt[2]]]` | `-1 - Sqrt[2]` | yes | 2→1 | 0.016 |
| C02 | `Strad[(-1 + 2*I*Sqrt[2])^(3/2)]` | `5 - I*Sqrt[2]` *PossibleZeroQ::ztest1* | **NO** | 2→1 | 0.396 |
| C03 | `Strad[Sqrt[-1 + 2*I*Sqrt[2]]* Sqrt[-2 + 2*I*Sqrt[3]]]` | `(-I + Sqrt[2])*(-I + Sqrt[3])` *PossibleZeroQ::ztest1* | **NO** | 2→1 | 0.128 |
| C04 | `Strad[(-3 - 2*Sqrt[2])^(3/2)]` | `(-I)*(7 + 5*Sqrt[2])` | yes | 2→1 | 0.013 |
| C05 | `Strad[Sqrt[-2 - 2*Sqrt[2]]*Sqrt[ -3 - 2*Sqrt[2]]]` | `-Sqrt[2*(7 + 5*Sqrt[2])]` | yes | 2→2 | 0.065 |
| C06 | `Strad[Sqrt[-3 - 2*Sqrt[2]]*Sqrt[ -3 - 2*Sqrt[2]]]` | `-3 - 2*Sqrt[2]` | yes | 1→1 | 0.009 |
| C07 | `Strad[(-1 + 2*I*Sqrt[2])^(5/2)]` | `1 - (11*I)*Sqrt[2]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.034 |
| C08 | `Strad[(-5 - 2*Sqrt[6])^(3/2)]` | `(-I)*(11*Sqrt[2] + 9*Sqrt[3])` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.045 |

**The engine called directly (Table 7).** D01, D04, D05 reproduce the branch loss at the engine level. D02 (multiplier 0) prints "success" and returns the unevaluated `{}[[1]]`; D10 (an inexact multiplier) returns `Outer[][[1]]`; D11 ($\pi$) is harmless. D06 versus D07 is R2's equal-degree gate: with $\{1, m\}$ the trial $m$ is skipped because its degree equals the best degree, with $\{m\}$ alone it is tried and "succeeds" with an uglier expression of the same depth (U9, U12). D08 shows that `Extension -> Automatic` copes with a nested radicand but that single-level mode cannot find $\sqrt{3+2\sqrt{3+2\sqrt{2}}} = \sqrt{5+2\sqrt{2}}$; it tries 12 multipliers and gives up (compare A28). D12 shows that a "forced" list does not prevent generated batches. D14 shows that the imaginary multiplier $i$ works.

Table 7: Original core `denest11` called directly with forced multipliers.

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| D01 | `denest11[-Sqrt[3 + 2*Sqrt[2]], {1}]` | `1 + Sqrt[2]` | **NO** | 2→1 | 0.005 |
| D02 | `denest11[Sqrt[3 + 2*Sqrt[2]], {0}]` | `{}[[1]]` *Power::infy, Part::partw* | **NO** | 2→0 | 0.089 |
| D03 | `denest11[Sqrt[3 + 2*Sqrt[2]], {}]` | `Sqrt[3 + 2*Sqrt[2]]` | yes | 2→2 | 0.001 |
| D04 | `denest11[(-1 + 2*I*Sqrt[2])^(3/2) , {1}]` | `5 - I*Sqrt[2]` | **NO** | 2→1 | 0.010 |
| D05 | `denest11[Sqrt[-1 + 2*I*Sqrt[2]]* Sqrt[-2 + 2*I*Sqrt[3]], {1}]` | `(-I + Sqrt[2])*(-I + Sqrt[3])` *PossibleZeroQ::ztest1* | **NO** | 2→1 | 0.026 |
| D06 | `denest11[Sqrt[5 + 2*Sqrt[6]], {1, 6 - 2*Sqrt[6] - 2*Sqrt[2] + 2* Sqrt[3]}]` | `Sqrt[5 + 2*Sqrt[6]]` | yes | 2→2 | 0.012 |

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| D07 | `denest11[Sqrt[5 + 2*Sqrt[6]], {6 - 2*Sqrt[6] - 2*Sqrt[2] + 2*Sqrt[3]}]` | `(1 + Sqrt[2] + Sqrt[3])/Sqrt[2*(3 - Sqrt[2] + Sqrt[3] - Sqrt[6])]` *PossibleZeroQ::ztest1* | yes | 2→2 | 0.056 |
| D08 | `denest11[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]]` | `Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]` | yes | 3→3 | 0.100 |
| D09 | `denest11[2 + Sqrt[3]]` | `2 + Sqrt[3]` | yes | 1→1 | 0.000 |
| D10 | `denest11[Sqrt[3 + 2*Sqrt[2]], {2.}]` | `Outer[][[1]]` *MinimalPolynomial::nalg, Outer::heads, Outer::argm, Part::partw* | **NO** | 2→0 | 0.238 |
| D11 | `denest11[Sqrt[3 + 2*Sqrt[2]], {Pi}]` | `Sqrt[3 + 2*Sqrt[2]]` *MinimalPolynomial::nalg* | yes | 2→2 | 0.004 |
| D12 | `denest11[Sqrt[5 + 2*Sqrt[6]], {1, Sqrt[5]}]` | `Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.014 |
| D13 | `denest11[Sqrt[3 + 2*Sqrt[2]], {-1}]` | `Sqrt[3 + 2*Sqrt[2]]` | yes | 2→2 | 0.005 |
| D14 | `denest11[Sqrt[3 + 2*Sqrt[2]], {I}]` | `1 + Sqrt[2]` *PossibleZeroQ::ztest1* | yes | 2→1 | 0.032 |

**Symbolic input (Table 8).** $\sqrt{z^2} \mapsto z$ and $\sqrt{-x-y} \mapsto i\sqrt{x+y}$ are the branch-unsafe rewrites of U2 reaching the public interface. E07 is R2's marker collision: `Strad[f[1], False, Identity]` returns `1`. E08 shows what a user function named `f` wrapped around a radical provokes: a cascade of seven different messages.

Table 8: Original program on symbolic input.

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| E01 | `Strad[Sqrt[z^2]]` | `z` | – | 1→0 | 0.001 |
| E02 | `Strad[Sqrt[t + t^2]]` | `Sqrt[t*(1 + t)]` | – | 1→1 | 0.002 |
| E03 | `Strad[Sqrt[-x - y]]` | `I*Sqrt[x + y]` | – | 1→1 | 0.004 |
| E04 | `Strad[Sqrt[a + b]]` | `Sqrt[a + b]` | – | 1→1 | 0.002 |
| E05 | `Strad[x]` | `x` | – | 0→0 | 0.000 |
| E06 | `Strad[Sqrt[x]*Sqrt[3 + 2*Sqrt[2]]]` | `(1 + Sqrt[2])*Sqrt[x]` | – | 2→1 | 0.010 |
| E07 | `Strad[f[1], False, Identity]` | `1` | – | 0→0 | 0.001 |
| E08 | `Strad[f[Sqrt[3 + 2*Sqrt[2]]]]` | `f[Sqrt[3 + 2*Sqrt[2]]]` *MinimalPolynomial::nalg, General::stop, Discriminant::poly2, FactorInteger::exact, Part::partd, Union::heads, Part::pkspec1* | – | 2→2 | 0.363 |

**Helpers (Tables 9 and 10).** `Factorc` preserves the value on 18 of 20 numeric probes; H05 is the numeric counterexample of Proposition 3.5(2): the result $i\sqrt{(-1)^{1/3}+\sqrt{2}}$ is the negative of the input. Note that `Factorc[Sqrt[t + t^2]]` (H02) returns `Sqrt[t (1 + t)]`, so R1's probe

does not trigger the defect. `RationalizeDenominator` is correct on every numeric denominator (R01–R10); R05 confirms R2's prediction that a numerator named `x` is erased (result 0), and R11 shows the helper emitting three messages on a symbolic denominator. S01 shows the comparator reversing a list; S02 the out-of-range slice; S04–S06 that `PossibleZeroQ` correctly rejects three very close non-zero differences, so U3 is a matter of unreliable *acceptance*, not of easily demonstrable false positives.

Table 9: `Factorc` on symbolic and numeric inputs. "Equal" is exact equality with the input.

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| H01 | `Factorc[Sqrt[z^2]]` | `z` | – | 0→0 | 0.000 |
| H02 | `Factorc[Sqrt[t + t^2]]` | `Sqrt[t*(1 + t)]` | – | 1→1 | 0.000 |
| H03 | `Factorc[Sqrt[-x - y]]` | `I*Sqrt[x + y]` | – | 1→1 | 0.001 |
| H04 | `Factorc[Sqrt[-2 - 2*Sqrt[2]]]` | `I*Sqrt[2*(1 + Sqrt[2])]` | yes | 2→2 | 0.001 |
| H05 | `Factorc[Sqrt[-(Sqrt[2] + (-1)^(1/3))]]` | `I*Sqrt[(-1)^(1/3) + Sqrt[2]]` | **NO** | 2→2 | 0.001 |
| H06 | `Factorc[Sqrt[(1 + I)*(3 + 2*Sqrt[2])]]` | `Sqrt[(1 + I)*(3 + 2*Sqrt[2])]` | yes | 2→2 | 0.001 |
| H07 | `Factorc[Sqrt[-1 - I]]` | `Sqrt[-1 - I]` | yes | 1→1 | 0.000 |
| H08 | `Factorc[Sqrt[-2 - 2*I*Sqrt[3]]]` | `Sqrt[2*(-1 - I*Sqrt[3])]` | yes | 2→2 | 0.001 |
| H09 | `Factorc[Sqrt[(-1 - I*Sqrt[3])*(1 + Sqrt[2])]]` | `Sqrt[(-I)*(1 + Sqrt[2])*(-I + Sqrt[3])]` | yes | 2→2 | 0.001 |
| H10 | `Factorc[(-3 - 2*Sqrt[2])^(1/3)]` | `(-3 - 2*Sqrt[2])^(1/3)` | yes | 2→2 | 0.000 |
| H11 | `Factorc[((-1 + I)*(3 + 2*Sqrt[2]))^(1/3)]` | `((-1 + I)*(3 + 2*Sqrt[2]))^(1/3)` | yes | 2→2 | 0.001 |
| H12 | `Factorc[Sqrt[2] + Sqrt[3]]` | `Sqrt[2] + Sqrt[3]` | yes | 1→1 | 0.001 |
| H13 | `Factorc[(1 + Sqrt[2])^2]` | `(1 + Sqrt[2])^2` | yes | 1→1 | 0.000 |
| H14 | `Factorc[Sqrt[8 + 4*Sqrt[3]]]` | `2*Sqrt[2 + Sqrt[3]]` | yes | 2→2 | 0.001 |
| H15 | `Factorc[Sqrt[-8 - 4*Sqrt[3]]]` | `(2*I)*Sqrt[2 + Sqrt[3]]` | yes | 2→2 | 0.001 |
| H16 | `Factorc[Sqrt[(-1)^(1/3)*(3 + 2*Sqrt[2])]]` | `(-1)^(1/6)*Sqrt[3 + 2*Sqrt[2]]` | yes | 2→2 | 0.001 |
| H17 | `Factorc[Sqrt[(-1)^(2/3)*(3 + 2*Sqrt[2])]]` | `(-1)^(1/3)*Sqrt[3 + 2*Sqrt[2]]` | yes | 2→2 | 0.001 |
| H18 | `Factorc[Sqrt[x^4]]` | `x^2` | – | 0→0 | 0.000 |
| H19 | `Factorc[(x^2)^(1/3)]` | `x^(2/3)` | – | 1→1 | 0.000 |
| H20 | `Factorc[Sqrt[4*x^2 + 4*x^3]]` | `2*Sqrt[x^2*(1 + x)]` | – | 1→1 | 0.001 |
| H21 | `Factorc[Sqrt[-3 - 2*Sqrt[2]]]` | `I*Sqrt[3 + 2*Sqrt[2]]` | yes | 2→2 | 0.001 |
| H22 | `Factorc[Sqrt[(-1 + I*Sqrt[3])*(-1 + I*Sqrt[3])]]` | `1 - I*Sqrt[3]` | yes | 1→1 | 0.000 |
| H23 | `Factorc[Sqrt[-(1 + Sqrt[2]) - (1 + Sqrt[2])*I*Sqrt[3]]]` | `Sqrt[(-I)*(1 + Sqrt[2])*(-I + Sqrt[3])]` | yes | 2→2 | 0.001 |

Table 10: Helper-level probes: `RationalizeDenominator`, marker collisions, nesting helpers, comparator, slice, `PossibleZeroQ`.

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| R01 | `RationalizeDenominator[1/(1 + Sqrt[2])]` | `-1 + Sqrt[2]` | yes | 1→1 | 0.000 |
| R02 | `RationalizeDenominator[1/(Sqrt[2] + Sqrt[3])]` | `-Sqrt[2] + Sqrt[3]` | yes | 1→1 | 0.002 |
| R03 | `RationalizeDenominator[1/(1 + I*Sqrt[2])]` | `(1 - I*Sqrt[2])/3` | yes | 1→1 | 0.001 |
| R04 | `RationalizeDenominator[(1 + Sqrt[2])/(Sqrt[2] - Sqrt[3])]` | `-2 - Sqrt[2] - Sqrt[3] - Sqrt[6]` | yes | 1→1 | 0.002 |
| R05 | `RationalizeDenominator[x/(1 + Sqrt[2])]` | `0` | yes | 0→0 | 0.000 |
| R06 | `RationalizeDenominator[1/2 + 1/Sqrt[2]]` | `1/2 + 1/Sqrt[2]` | yes | 1→1 | 0.000 |
| R07 | `RationalizeDenominator[1/Sqrt[2]]` | `1/Sqrt[2]` | yes | 1→1 | 0.000 |
| R08 | `RationalizeDenominator[1/(1 + 2^(1/3))]` | `(1 - 2^(1/3) + 2^(2/3))/3` | yes | 1→1 | 0.001 |
| R09 | `RationalizeDenominator[1/Sqrt[1 + Sqrt[2]]]` | `-Sqrt[1 + Sqrt[2]] + Sqrt[2*(1 + Sqrt[2])]` | yes | 2→2 | 0.001 |
| R10 | `RationalizeDenominator[0]` | `0` | yes | 0→0 | 0.000 |
| R11 | `RationalizeDenominator[1/x]` | `-(PolynomialQuotient[0, 0, 0]/MinimalPolynomial[x][0])` *MinimalPolynomial::nalg, General::stop, PolynomialQuotient::ivar* | – | 0→0 | 0.008 |
| M01 | `DenestRadicals3[f[1], False, Identity]` | `1` | – | 0→0 | 0.000 |
| M02 | `DenestRadicals3[g[1], False, Identity]` | `g[1]` | – | 0→0 | 0.000 |
| M03 | `DenestRadicals3[f[Sqrt[3 + 2*Sqrt[2]]], False, Identity]` | `f[Sqrt[3 + 2*Sqrt[2]]]` *General::newsym* | – | 2→2 | 0.007 |
| N01 | `maxnestdepth[Sqrt[3 + 2*Sqrt[2]], MatchQ[#1, _^_Rational] & ]` | `{{Sqrt[3 + 2*Sqrt[2]], 2}}` | – | 2→2 | 0.000 |
| N02 | `maxnestdepth[(1 + 2^(1/3))^3, MatchQ[#1, _^_Rational] & ]` | `{{2^(1/3), 1}}` | – | 1→1 | 0.000 |
| N03 | `LCM @@ (maxnestdepth[(1 + 2^(1/3))^3, MatchQ[#1, _^_Rational] & ] /. {(base_)^(exp_), _} :> Denominator[exp])` | `3` | yes | 0→0 | 0.000 |
| N04 | `maxnestdepth[Sqrt[1 + 2^(1/3)]*(1 + Sqrt[2])^(1/3), MatchQ[#1, _^_Rational] & ]` | `{{Sqrt[1 + 2^(1/3)], 2}, {(1 + Sqrt[2])^(1/3), 2}}` | – | 2→2 | 0.000 |

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| N05 | `nestdepthsort[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]], MatchQ[#1, _^ _Rational] & ]` | `{{{Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]], 3}}, {{3 + 2*Sqrt[3 + 2* Sqrt[2]], 2}, {2*Sqrt[3 + 2* Sqrt[2]], 2}, {Sqrt[3 + 2*Sqrt[ 2]], 2}}, {{3 + 2*Sqrt[2], 1}, {2*Sqrt[2], 1}, {Sqrt[2], 1}}, {{3, 0}, {2, 0}, {3, 0}, {2, 0}, {2, 0}, {1/2, 0}, {1/2, 0}, {1/2, 0}}}` | – | 3→3 | 0.000 |
| N06 | `maxnestdepth[Sqrt[3 + 2*Sqrt[2]] + (5 + 2*Sqrt[6])^(1/3), MatchQ[ #1, _^_Rational] & ]` | `{{Sqrt[3 + 2*Sqrt[2]], 2}, {(5 + 2*Sqrt[6])^(1/3), 2}}` | – | 2→2 | 0.000 |
| S01 | `Sort[{{2, 1}, {3, 1}, {5, 1}, {7, 1}, {11, 1}, {13, 1}, {2003, 1}}, #1[[1]] <= #1[[2]] & ]` | `{{2003, 1}, {13, 1}, {11, 1}, {7, 1}, {5, 1}, {3, 1}, {2, 1}}` | – | 0→0 | 0.000 |
| S02 | `Module[{p = {2, 3, 5, 7, 11, 0}}, DeleteCases[p, 0][[1 ;; Min[10, Length[p]]]]]` | `{2, 3, 5, 7, 11}[[1 ;; 6]]` *Part::take* | – | 0→0 | 0.000 |
| S03 | `ComplexitySort[{6, Sqrt[2], 2, 3, 1 + Sqrt[2], 10, -3}]` | `{2, 3, -3, 6, 10, Sqrt[2], 1 + Sqrt[2]}` | – | 1→1 | 0.000 |
| S04 | `PossibleZeroQ[Sqrt[2] - FromContinuedFraction[ ContinuedFraction[Sqrt[2], 400]] ]` | `False` | – | 0→0 | 0.000 |
| S05 | `PossibleZeroQ[Sqrt[2] - FromContinuedFraction[ ContinuedFraction[Sqrt[2], 400]], Method -> "ExactAlgebraics"]` | `False` *N::meprec* | – | 0→0 | 0.001 |
| S06 | `PossibleZeroQ[Sqrt[2] - Root[#1^ 2 - 2 + 10^(-100) & , 2]]` | `False` | – | 0→0 | 0.001 |
| S07 | `DeleteDuplicates[{1 + Sqrt[2], Sqrt[3 + 2*Sqrt[2]], 1 + Sqrt[2] }]` | `{1 + Sqrt[2], Sqrt[3 + 2*Sqrt[2] ]}` | – | 2→2 | 0.000 |

**Wrapper-level numeric branch errors and odd inputs (Table 11).** J01 and J02 are the most important new findings: a purely numeric input, $\sqrt{-(\sqrt{2}+(-1)^{1/3})}$, alone or inside a sum, is returned with the wrong sign, silently. J06 is the cube-root analogue. K14 shows that a *list* of two radicals is turned into $(3+2\sqrt{2})^{(\sqrt{2}+\sqrt{3})/2}$. K04 shows that the ugly BFHT output is a fixed point of the program. K12: inexact input is returned as a machine number.

Table 11: Original program: wrapper-level numeric branch errors caused by `Factorc` (J) and miscellaneous inputs (K).

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| J01 | `Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3) )]]` | `I*Sqrt[(-1)^(1/3) + Sqrt[2]]` | **NO** | 2→2 | 0.097 |

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| J02 | `Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3))] + 1]` | `1 + I*Sqrt[(-1)^(1/3) + Sqrt[2]]` | **NO** | 2→2 | 0.101 |
| J03 | `Factorc[Sqrt[(-(1 + Sqrt[2]))*((1 + I*Sqrt[3])/2)]]` | `Sqrt[(-1/2*I)*(1 + Sqrt[2])*(-I + Sqrt[3])]` | yes | 2→2 | 0.004 |
| J04 | `Strad[Sqrt[(-(1 + Sqrt[2]))*((1 + I*Sqrt[3])/2)]]` | `Sqrt[(-1/2*I)*(1 + Sqrt[2])*(-I + Sqrt[3])]` | yes | 2→2 | 0.313 |
| J05 | `Strad[Sqrt[-3 - 2*Sqrt[2] - I*(3 + 2*Sqrt[2])*Sqrt[3]]]` | `((2 + Sqrt[2])*(1 - I*Sqrt[3]))/2` | yes | 1→1 | 0.026 |
| J06 | `Strad[(-(Sqrt[2] + (-1)^(1/3)))^(1/3)]` | `(-1)^(1/3)*((-1)^(1/3) + Sqrt[2])^(1/3)` | **NO** | 2→2 | 0.792 |
| J07 | `Strad[Sqrt[-Sqrt[2] - I*Sqrt[3]]]` | `Sqrt[-Sqrt[2] - I*Sqrt[3]]` | yes | 2→2 | 0.109 |
| J08 | `Strad[Sqrt[-1 - I*Sqrt[3] - Sqrt[2] - I*Sqrt[6]]]` | `Sqrt[(-I)*(1 + Sqrt[2])*(-I + Sqrt[3])]` | yes | 2→2 | 0.450 |
| K01 | `Strad[Sqrt[28^(1/3) - 3]]` | `(-1 - 2^(2/3)*7^(1/3) + 2^(1/3)*7^(2/3))/3` | yes | 1→1 | 0.018 |
| K02 | `Strad[(1 + 2^(1/3))^3]` | `(1 + 2^(1/3))^3` | yes | 1→1 | 0.002 |
| K03 | `denest11[(1 + 2^(1/3))^3]` | `3*(1 + 2^(1/3) + 2^(2/3))` | yes | 1→1 | 0.006 |
| K04 | `Strad[Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]], True]]` | `(Sqrt[319 - 58*Sqrt[29]] - 10*Sqrt[11 - 2*Sqrt[29]] + Sqrt[5]*(-16 + 3*Sqrt[29]))/(-5 + Sqrt[29] - Sqrt[55 - 10*Sqrt[29]])` *PossibleZeroQ::ztest1* | yes | 2→2 | 0.370 |
| K05 | `Strad[Sqrt[2 + Sqrt[3]]*Sqrt[2 - Sqrt[3]]]` | `1` | yes | 0→0 | 0.001 |
| K06 | `Strad[Sqrt[Sqrt[2] - 1]*Sqrt[Sqrt[2] + 1]]` | `1` | yes | 0→0 | 0.000 |
| K07 | `Strad[Sqrt[3 + 2*Sqrt[2]]^(1/3)]` | `(1 + Sqrt[2])^(1/3)` | yes | 2→2 | 0.010 |
| K08 | `Strad[(3 + 2*Sqrt[2])^(1/6)]` | `(1 + Sqrt[2])^(1/3)` | yes | 2→2 | 0.012 |
| K09 | `Strad[Sqrt[5 + 2*Sqrt[6]]/Sqrt[3 + 2*Sqrt[2]]]` | `2 - Sqrt[2] - Sqrt[3] + Sqrt[6]` | yes | 1→1 | 0.024 |
| K10 | `Strad[Exp[I*(Pi/7)]*Sqrt[3 + 2*Sqrt[2]]]` | `(-1)^(1/7)*(1 + Sqrt[2])` | yes | 1→1 | 0.010 |
| K11 | `Strad[Sqrt[3 + 2*Sqrt[2]] + Pi]` | `1 + Sqrt[2] + Pi` | yes | 1→1 | 0.010 |
| K12 | `Strad[Sqrt[3.  + 2*Sqrt[2]]]` | `2.414213562373095` | yes | 0→0 | 0.000 |
| K13 | `Strad[Sqrt[2 + Sqrt[3]], "yes"]` | `(1 + Sqrt[3])/Sqrt[2]` | yes | 1→1 | 0.013 |
| K14 | `Strad[{Sqrt[3 + 2*Sqrt[2]], Sqrt[5 + 2*Sqrt[6]]}]` | `(3 + 2*Sqrt[2])^((Sqrt[2] + Sqrt[3])/2)` | yes | 1→1 | 0.019 |
| K15 | `Strad[Sqrt[3 + 2*Sqrt[2]] == 1 + Sqrt[2]]` | `True` | – | 0→0 | 0.002 |
| K16 | `Strad[Sqrt[9 + 4*Sqrt[5]]]` | `2 + Sqrt[5]` | yes | 1→1 | 0.009 |
| K17 | `Strad[Sqrt[(3 + 2*Sqrt[2])/(5 + 2*Sqrt[6])]]` | `-2 - Sqrt[2] + Sqrt[3] + Sqrt[6]` | yes | 1→1 | 0.024 |
| K18 | `Strad[Sqrt[3 + 2*Sqrt[2]]*Sqrt[x]]` | `(1 + Sqrt[2])*Sqrt[x]` | yes | 1→1 | 0.008 |
| K19 | `Strad[Sqrt[Sqrt[2] + 1]^3]` | `(1 + Sqrt[2])^(3/2)` | yes | 2→2 | 0.028 |
| K20 | `Strad[Sqrt[2 + Sqrt[3]]*Sqrt[3 + 2*Sqrt[2]]]` | `((2 + Sqrt[2])*(1 + Sqrt[3]))/2` | yes | 1→1 | 0.017 |

**Higher-order roots, `PossibleZeroQ`, prime processing (Table 12).** L04, a four-term square root, is denested to a correct but enormous quotient (rationalizing its denominator and simplifying gives $\sqrt{2}+\sqrt{3}+\sqrt{5}+\sqrt{7}$; the corrected version does this). L14, a tenth root, takes 20 s and emits `Ceiling::meprec` because `Ceiling[Log[1000]/Log[10]]` does not evaluate (R01–R04 in Table 13). N01–N04 compare the prime processing with the original and the corrected comparator on the same factor lists: the original reverses the list and never prunes; the corrected one sorts, prunes, and then hits the stale-length `Part::take` (U6). N05/N06 materialize the $2^{10}$ and $10^5$ divisor lists that the "cap" permits.

Table 12: Original program: higher-order roots and timing (L), `PossibleZeroQ` on actual candidate differences (M), prime processing with the original and the corrected comparator (N).

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| L01 | `Strad[(239 + 169*Sqrt[2])^(1/7)]` | `1 + Sqrt[2]` | yes | 1→1 | 0.008 |
| L02 | `Strad[(2 + Sqrt[2])^(1/5)]` | `2^(1/10)*(1 + Sqrt[2])^(1/5)` | yes | 2→2 | 0.198 |
| L03 | `Strad[(1 + Sqrt[2])^(1/7)]` | `(1 + Sqrt[2])^(1/7)` | yes | 2→2 | 0.494 |
| L04 | `Strad[Sqrt[17 + 2*Sqrt[6] + 2* Sqrt[10] + 2*Sqrt[14] + 2*Sqrt[ 15] + 2*Sqrt[21] + 2*Sqrt[35]]]` | `(6397*Sqrt[2] + 5316*Sqrt[3] + 4090*Sqrt[5] + 3432*Sqrt[7] + 1695*Sqrt[30] + 1423*Sqrt[42] + 1095*Sqrt[70] + 910*Sqrt[105])/( 1108 + 467*Sqrt[6] + 359*Sqrt[ 10] + 299*Sqrt[14] + 302*Sqrt[ 15] + 252*Sqrt[21] + 194*Sqrt[ 35] + 81*Sqrt[210])` | yes | 1→1 | 0.956 |
| L05 | `Strad[Expand[(1 + Sqrt[2] + Sqrt[3])^3]^(1/3)]` | `1 + Sqrt[2] + Sqrt[3]` | yes | 1→1 | 0.020 |
| L06 | `Strad[Expand[(Sqrt[2] + Sqrt[3]) ^4]^(1/4)]` | `Sqrt[5 + 2*Sqrt[6]]` | yes | 2→2 | 0.013 |
| L07 | `Strad[Sqrt[1000003 + 2*Sqrt[ 999983]]]` | `Sqrt[1000003 + 2*Sqrt[999983]]` | yes | 2→2 | 0.215 |
| L08 | `Strad[Sqrt[2 + Sqrt[3] + Sqrt[5] ]]` | `Sqrt[2 + Sqrt[3] + Sqrt[5]]` | yes | 2→2 | 0.216 |
| L09 | `Strad[(5 + 2*Sqrt[6])^(1/6)]` | `(2 + Sqrt[6])^(1/3)/2^(1/6)` | yes | 2→2 | 2.256 |
| L10 | `Strad[(11*Sqrt[2] + 9*Sqrt[3])^( 1/3)]` | `Sqrt[2] + Sqrt[3]` | yes | 1→1 | 0.015 |
| L11 | `Strad[Expand[(2^(1/3) + 3^(1/3)) ^3]^(1/3)]` | `2^(1/3) + 3^(1/3)` | yes | 1→1 | 0.133 |
| L12 | `Strad[Sqrt[Expand[(1 + 2^(1/3) + 3^(1/3))^2]]]` | `1 + 2^(1/3) + 3^(1/3)` | yes | 1→1 | 0.006 |
| L13 | `Strad[Expand[(1 + 2^(1/5))^5]^(1/ 5)]` | `1 + 2^(1/5)` | yes | 1→1 | 0.023 |
| L14 | `Strad[Sqrt[5 + 2*Sqrt[6]]^(1/5)]` | `-(((-1)^(4/5)*(-2 - Sqrt[6])^(1/ 5))/2^(1/10))` *Ceiling::meprec* | yes | 2→2 | 20.350 |
| M01 | `PossibleZeroQ[Sqrt[3/2] + 1/ Sqrt[2] - Sqrt[2 + Sqrt[3]]]` | `True` | – | 0→0 | 0.000 |

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| M02 | `PossibleZeroQ[Sqrt[3/2] + 1/ Sqrt[2] - Sqrt[2 + Sqrt[3]], Method -> "ExactAlgebraics"]` | `True` | – | 0→0 | 0.000 |
| M03 | `RootReduce[Sqrt[3/2] + 1/Sqrt[2] - Sqrt[2 + Sqrt[3]]]` | `0` | yes | 0→0 | 0.014 |
| M04 | `PossibleZeroQ[-Sqrt[3/2] - 1/ Sqrt[2] - Sqrt[2 + Sqrt[3]]]` | `False` | – | 0→0 | 0.000 |
| M05 | `PossibleZeroQ[(1 + I*Sqrt[2])*(1 + I*Sqrt[3]) - Sqrt[Expand[(1 + I*Sqrt[2])^2*(1 + I*Sqrt[3])^2]] ]` | `False` | – | 0→0 | 0.000 |
| M06 | `PossibleZeroQ[(-(1 + I*Sqrt[2]))* (1 + I*Sqrt[3]) - Sqrt[Expand[(1 + I*Sqrt[2])^2*(1 + I*Sqrt[3])^2] ]]` | `True` | – | 0→0 | 0.000 |
| N01 | `primeProc[facs, #1[[1]] <= #1[[ 2]] & , 2]` | `{{2, 2003, 11, 7, 5, 3}, {2, 2003, 11, 7, 5, 3}}` | – | 0→0 | 0.000 |
| N02 | `primeProc[facs, #1[[1]] <= #2[[ 1]] & , 2]` | `{{2, 3, 5, 7, 11, 2003}, {2, 3, 5, 7, 11}[[1 ;; 6]]}` *Part::take* | – | 0→0 | 0.000 |
| N03 | `primeProc[{{2, 14}, {3, 2}, {5, 1}, {7, 1}, {11, 1}, {13, 1}, {17, 1}, {19, 1}, {23, 1}, {29, 1}, {31, 1}}, #1[[1]] <= #1[[2]] & , 2]` | `{{2, 31, 29, 23, 19, 17, 13, 11, 7, 5, 3}, {2, 31, 29, 23, 19, 17, 13, 11, 7, 5}}` | – | 0→0 | 0.000 |
| N04 | `primeProc[{{2, 14}, {3, 2}, {5, 1}, {7, 1}, {11, 1}, {13, 1}, {17, 1}, {19, 1}, {23, 1}, {29, 1}, {31, 1}}, #1[[1]] <= #2[[1]] & , 2]` | `{{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31}, {2, 3, 5, 7, 11, 13, 17, 19, 23, 29}}` | – | 0→0 | 0.000 |
| N05 | `Length[Divisors[(2*3*5*7*11*13* 17*19*23*29)^(2 - 1)]]` | `1024` | yes | 0→0 | 0.000 |
| N06 | `Length[Divisors[(2*3*5*7*11)^(10 - 1)]]` | `100000` | yes | 0→0 | 0.039 |

**Negative bases and output quality (Table 13).** Q01, Q02, Q07, Q08: for $(p + q\sqrt{c})^3 < 0$ the principal cube root is $e^{i\pi/3}|p + q\sqrt{c}|$; the program's "solutions" are equal to the input but contain $\sqrt{9 \pm 6\sqrt{2}}$ and similar denestable surds, so the depth does not decrease (U23). Q03–Q06 (fourth roots and positive bases) are fine.

Table 13: Original program: roots of negative bases (Q), the `Ceiling[Log]` issue (R), output quality for a four-term square root (S).

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| Q01 | `Strad[Expand[(-1 - Sqrt[2])^3]^(1/3)]` | `((-1)^(2/3)*(1 + Sqrt[2] - I*Sqrt[9 + 6*Sqrt[2]]))/2` | yes | 2→2 | 0.021 |
| Q02 | `Strad[Expand[(1 - Sqrt[2])^3]^(1/3)]` | `((-1)^(2/3)*(-1 + Sqrt[2] - I*Sqrt[9 - 6*Sqrt[2]]))/2` | yes | 2→2 | 0.019 |
| Q03 | `Strad[Expand[(-1 - Sqrt[2])^4]^(1/4)]` | `1 + Sqrt[2]` | yes | 1→1 | 0.008 |
| Q04 | `Strad[Expand[(1 - Sqrt[2])^4]^(1/4)]` | `-1 + Sqrt[2]` | yes | 1→1 | 0.013 |
| Q05 | `Strad[Expand[(2 - 3*Sqrt[3])^4]^(1/4)]` | `-2 + 3*Sqrt[3]` | yes | 1→1 | 0.015 |
| Q06 | `Strad[Expand[(3 - Sqrt[2])^3]^(1/3)]` | `3 - Sqrt[2]` | yes | 1→1 | 0.014 |
| Q07 | `Strad[Expand[(2 - 3*Sqrt[2])^3]^(1/3)]` | `-(((-1)^(2/3)*(-3 + Sqrt[2] + I*Sqrt[33 - 18*Sqrt[2]]))/Sqrt[2])` | yes | 2→2 | 0.023 |
| Q08 | `Strad[(-7 - 5*Sqrt[2])^(1/3)]` | `((-1)^(2/3)*(1 + Sqrt[2] - I*Sqrt[9 + 6*Sqrt[2]]))/2` | yes | 2→2 | 0.017 |
| Q09 | `denest11[(7 - 5*Sqrt[2])^(1/3)]` | `((-1)^(2/3)*(-1 + Sqrt[2] - I*Sqrt[9 - 6*Sqrt[2]]))/2` | yes | 2→2 | 0.013 |
| R01 | `Ceiling[Log[1000]/Log[10]]` | `Ceiling[Log[1000]/Log[10]]` *Ceiling::meprec* | yes | 0→0 | 0.001 |
| R02 | `Max[5, Ceiling[Log[1000]/Log[10]]]` | `5` *Ceiling::meprec* | yes | 0→0 | 0.001 |
| R03 | `Min[Max[5, Ceiling[Log[1000]/Log[10]]], 3]` | `3` *Ceiling::meprec* | yes | 0→0 | 0.000 |
| R04 | `{2, 3, 5}[[1 ;; Min[Max[5, Ceiling[Log[1000]/Log[10]]], 3]]]` | `{2, 3, 5}` *Ceiling::meprec* | – | 0→0 | 0.000 |
| R05 | `Ceiling[Log[1000]/Log[100]]` | `2` | yes | 0→0 | 0.000 |
| R06 | `Ceiling[Log[1000]/Log[1000]]` | `1` | yes | 0→0 | 0.000 |
| R07 | `Ceiling[Log[1000]/Log[6]]` | `4` | yes | 0→0 | 0.000 |
| S01 | `Strad[Sqrt[17 + 2*Sqrt[6] + 2*Sqrt[10] + 2*Sqrt[14] + 2*Sqrt[15] + 2*Sqrt[21] + 2*Sqrt[35]]]` | `(6397*Sqrt[2] + 5316*Sqrt[3] + 4090*Sqrt[5] + 3432*Sqrt[7] + 1695*Sqrt[30] + 1423*Sqrt[42] + 1095*Sqrt[70] + 910*Sqrt[105])/(1108 + 467*Sqrt[6] + 359*Sqrt[10] + 299*Sqrt[14] + 302*Sqrt[15] + 252*Sqrt[21] + 194*Sqrt[35] + 81*Sqrt[210])` | yes | 1→1 | 2.355 |

### 5.3 Randomized families

Ten structured families with a fixed seed (`SeedRandom[20260905]`), $p, q, r \in \{\pm 1, \ldots, \pm 7\}$ and $c, d$ square-free in $\{2, \ldots, 17\}$, were run through `Strad` of both versions with a 60 s limit per input. Each result was classified by exact comparison with the input. Table 14 summarizes; "wrong" means the

returned value differs from the input.

The F3 family is a control: with $a \in [-30, 30]$ random, only 6 of 40 radicands satisfy the square criterion of Proposition 3.6, and both versions denest exactly those 6. The 8 "equal, rewritten" outputs of the original in F3 are the final `Simplify` at work (U20); the corrected version returns such inputs unchanged. The 17 rewritten outputs in F2 and 8 in F7 are the negative-base cases of Table 13.

### 5.4 The reviews' regression suites

Table 15 lists the outcome of each review's own Wolfram tests on the original program. Every "expected to fail" contract fails, every control passes, and no test aborted.

### 5.5 The reviews' proposed kernels

The three one-trial components (R1 `VerifiedMultiplierTrial`, R2 `CertifiedMultiplierStep`, R3 `TryDenestCandidate`) were loaded and called on eleven common cases. All three certify the correct branch for the negative target, the complex 3/2 power, the complex product and the cube-root case with $m = 9$; all three reject a zero multiplier, an inexact multiplier, root order 1 and a symbolic target with a structured `Failure`; all three report "no lower-degree GCD" for $\sqrt{5 + 2\sqrt{6}}$ with $m = 1$ and find $\sqrt{2} + \sqrt{3}$ with $m = 2$, and, because none of them has a degree gate, all three also find it with R2's equal-degree multiplier (returning the same ugly quotient as the original, D07). R3's `BestImprovement` policy correctly rejects that quotient as "not an improvement", but would accept the original's BFHT output (Table eftab:origA, A31), because its cost tuple puts radical depth before size: $(0, 2, 9, 69)$ beats $(0, 3, 4, 28)$. In other words the components work as their authors said, and they are single steps, not denesters: none contains a multiplier generator, a wrapper, or a search.

### 5.6 The corrected version

Tables 16–20 repeat the battery with `StradFixed.wl`. Every row is exactly equal to the true value; no row emits a message; `Print` keeps its `Protected` attribute. Beyond fixing every wrong value, the corrected version finds denestings the original does not: the BFHT example (A30/A31), the four-term square root as $\sqrt{2}+\sqrt{3}+\sqrt{5}+\sqrt{7}$ (L04), $(49+20\sqrt{6})^{1/4} = \sqrt{2}+\sqrt{3}$ (L06), the cube roots of negative bases (Q01–Q08) and the equal-degree multiplier (D06). It also returns inputs unchanged when it cannot improve them (B01, B08, B09, L02, L03, L09) instead of rewriting them, converts a `Root` input to $\sqrt{2} + \sqrt{3}$ (B07), maps over lists (K14), leaves symbolic hosts alone (E01–E05, E07) while still denesting numeric parts inside them (E06, E08), and rejects invalid multipliers without garbage (D02, D10, D11). The price is a factor of roughly two to three in time on the small inputs, almost entirely spent in `RootReduce` certification.

[1] R2's suite restores the attributes of `Print` itself after loading and then tests the recorded values, so this test measures the suite's own bookkeeping rather than the defect; R1's and R3's versions of the same test fail as they should.

Table 16: Corrected version on the classical identities (compare Table 4).

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| A01 | `Strad[Sqrt[3 + 2*Sqrt[2]]]` | `1 + Sqrt[2]` | yes | 2→1 | 0.103 |
| A02 | `Strad[Sqrt[5 + 2*Sqrt[6]]]` | `Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.055 |
| A03 | `Strad[Sqrt[2 + Sqrt[3]]]` | `Sqrt[3/2] + 1/Sqrt[2]` | yes | 2→1 | 0.057 |
| A04 | `Strad[Sqrt[11 + 6*Sqrt[2]]]` | `3 + Sqrt[2]` | yes | 2→1 | 0.030 |
| A05 | `Strad[Sqrt[3 + Sqrt[5]]]` | `1/Sqrt[2] + Sqrt[5/2]` | yes | 2→1 | 0.060 |
| A06 | `Strad[Sqrt[7 + 2*Sqrt[10]]]` | `Sqrt[2] + Sqrt[5]` | yes | 2→1 | 0.040 |
| A07 | `Strad[Sqrt[6 + 2*Sqrt[2] + 2* Sqrt[3] + 2*Sqrt[6]]]` | `1 + Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.093 |
| A08 | `Strad[Sqrt[12 + 2*Sqrt[6] + 2* Sqrt[14] + 2*Sqrt[21]]]` | `Sqrt[2] + Sqrt[3] + Sqrt[7]` | yes | 2→1 | 0.231 |
| A09 | `Strad[(Sqrt[5] - 2)^(1/3)]` | `(-1 + Sqrt[5])/2` | yes | 2→1 | 0.045 |
| A10 | `Strad[(2 + Sqrt[5])^(1/3)]` | `(1 + Sqrt[5])/2` | yes | 2→1 | 0.046 |
| A11 | `Strad[(10 + 6*Sqrt[3])^(1/3)]` | `1 + Sqrt[3]` | yes | 2→1 | 0.066 |
| A12 | `Strad[(2^(1/3) - 1)^(1/3)]` | `(1 - 2^(1/3) + 2^(2/3))/3^(2/3)` | yes | 2→1 | 0.337 |
| A13 | `Strad[(5^(1/3) - 4^(1/3))^(1/3)]` | `(-2^(2/3) + 5^(1/3))^(1/3)` | yes | 2→2 | 1.503 |
| A14 | `Strad[((11 + 5*Sqrt[5])/2)^(1/5) ]` | `(1 + Sqrt[5])/2` | yes | 2→1 | 0.045 |
| A15 | `Strad[Sqrt[1 + 2*2^(1/3) + 2^(2/ 3)]]` | `1 + 2^(1/3)` | yes | 2→1 | 0.019 |
| A16 | `Strad[(17 + 12*Sqrt[2])^(1/4)]` | `1 + Sqrt[2]` | yes | 2→1 | 0.042 |
| A17 | `Strad[(99 + 70*Sqrt[2])^(1/6)]` | `1 + Sqrt[2]` | yes | 2→1 | 0.044 |
| A18 | `Strad[(3 + 2*Sqrt[2])^(3/2)]` | `7 + 5*Sqrt[2]` | yes | 2→1 | 0.027 |
| A19 | `Strad[(3 + 2*Sqrt[2])^(-2^(-1))]` | `-1 + Sqrt[2]` | yes | 2→1 | 0.049 |
| A20 | `Strad[1/Sqrt[3 + 2*Sqrt[2]]]` | `-1 + Sqrt[2]` | yes | 2→1 | 0.042 |
| A21 | `Strad[Sqrt[3 + 2*Sqrt[2]] + Sqrt[5 + 2*Sqrt[6]]]` | `1 + 2*Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.043 |
| A22 | `Strad[Sqrt[3 + 2*Sqrt[2]]*Sqrt[5 + 2*Sqrt[6]]]` | `2 + Sqrt[2] + Sqrt[3] + Sqrt[6]` | yes | 2→1 | 0.094 |
| A23 | `Strad[Sqrt[2 + Sqrt[3]] + Sqrt[2 - Sqrt[3]]]` | `Sqrt[6]` | yes | 2→1 | 0.058 |
| A24 | `Strad[Sqrt[-1 + 2*I*Sqrt[2]]]` | `1 + I*Sqrt[2]` | yes | 2→1 | 0.049 |
| A25 | `Strad[Sqrt[2 + 2*I*Sqrt[3]]]` | `I + Sqrt[3]` | yes | 2→1 | 0.071 |
| A26 | `Strad[Sqrt[-3 - 2*Sqrt[2]]]` | `I*(1 + Sqrt[2])` | yes | 2→1 | 0.028 |
| A27 | `Strad[(3 + 2*Sqrt[2])^(1/4)]` | `Sqrt[1 + Sqrt[2]]` | yes | 2→2 | 0.070 |
| A28 | `Strad[Sqrt[3 + 2*Sqrt[3 + 2* Sqrt[2]]], True]` | `Sqrt[5 + 2*Sqrt[2]]` | yes | 3→2 | 0.090 |
| A29 | `Strad[Sqrt[3 + 2*Sqrt[3 + 2* Sqrt[2]]]]` | `Sqrt[3 + 2*(1 + Sqrt[2])]` | yes | 3→2 | 0.093 |
| A30 | `Strad[Sqrt[16 - 2*Sqrt[29] + 2* Sqrt[55 - 10*Sqrt[29]]], True]` | `Sqrt[5] + Sqrt[11 - 2*Sqrt[29]]` | yes | 3→2 | 1.165 |
| A31 | `Strad[Sqrt[16 - 2*Sqrt[29] + 2* Sqrt[55 - 10*Sqrt[29]]]]` | `Sqrt[5] + Sqrt[11 - 2*Sqrt[29]]` | yes | 3→2 | 0.531 |

Table 17: Corrected version on non-denestable inputs (B) and on the branch counterexamples (C).

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| B01 | `Strad[Sqrt[2 + Sqrt[2]]]` | `Sqrt[2 + Sqrt[2]]` | yes | 2→2 | 0.041 |
| B02 | `Strad[Sqrt[1 + Sqrt[2]]]` | `Sqrt[1 + Sqrt[2]]` | yes | 2→2 | 0.023 |
| B03 | `Strad[(1 + Sqrt[2])^(1/3)]` | `(1 + Sqrt[2])^(1/3)` | yes | 2→2 | 0.069 |
| B04 | `Strad[Sqrt[2]]` | `Sqrt[2]` | yes | 1→1 | 0.003 |
| B05 | `Strad[2]` | `2` | yes | 0→0 | 0.001 |
| B06 | `Strad[Sqrt[1 + I]]` | `Sqrt[1 + I]` | yes | 1→1 | 0.003 |
| B07 | `Strad[Root[#1^4 - 10*#1^2 + 1 & , 4]]` | `Sqrt[2] + Sqrt[3]` | yes | 0→1 | 0.017 |
| B08 | `Strad[Sqrt[Sqrt[2] + Sqrt[3]]]` | `Sqrt[Sqrt[2] + Sqrt[3]]` | yes | 2→2 | 0.080 |
| B09 | `Strad[(2 + 2*I*Sqrt[3])^(1/3)]` | `(2 + (2*I)*Sqrt[3])^(1/3)` | yes | 2→2 | 0.269 |
| C01 | `Strad[-Sqrt[3 + 2*Sqrt[2]]]` | `-1 - Sqrt[2]` | yes | 2→1 | 0.027 |
| C02 | `Strad[(-1 + 2*I*Sqrt[2])^(3/2)]` | `-5 + I*Sqrt[2]` | yes | 2→1 | 0.043 |
| C03 | `Strad[Sqrt[-1 + 2*I*Sqrt[2]]* Sqrt[-2 + 2*I*Sqrt[3]]]` | `-((-I + Sqrt[2])*(-I + Sqrt[3]))` | yes | 2→1 | 0.138 |
| C04 | `Strad[(-3 - 2*Sqrt[2])^(3/2)]` | `(-I)*(7 + 5*Sqrt[2])` | yes | 2→1 | 0.054 |
| C05 | `Strad[Sqrt[-2 - 2*Sqrt[2]]*Sqrt[ -3 - 2*Sqrt[2]]]` | `-Sqrt[2*(7 + 5*Sqrt[2])]` | yes | 2→2 | 0.050 |
| C06 | `Strad[Sqrt[-3 - 2*Sqrt[2]]*Sqrt[ -3 - 2*Sqrt[2]]]` | `-3 - 2*Sqrt[2]` | yes | 1→1 | 0.009 |
| C07 | `Strad[(-1 + 2*I*Sqrt[2])^(5/2)]` | `1 - (11*I)*Sqrt[2]` | yes | 2→1 | 0.045 |
| C08 | `Strad[(-5 - 2*Sqrt[6])^(3/2)]` | `(-I)*(11*Sqrt[2] + 9*Sqrt[3])` | yes | 2→1 | 0.091 |

Table 18: Corrected version: direct core calls with forced multipliers (D) and symbolic input (E).

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| D01 | `DenestCore[-Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {1}]` | `-1 - Sqrt[2]` | yes | 2→1 | 0.029 |
| D02 | `DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {0}]` | `1 + Sqrt[2]` | yes | 2→1 | 0.012 |
| D03 | `DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {}]` | `1 + Sqrt[2]` | yes | 2→1 | 0.006 |
| D04 | `DenestCore[(-1 + 2*I*Sqrt[2])^(3/ 2), "Multipliers" -> {1}]` | `-5 + I*Sqrt[2]` | yes | 2→1 | 0.028 |
| D05 | `DenestCore[Sqrt[-1 + 2*I*Sqrt[2] ]*Sqrt[-2 + 2*I*Sqrt[3]], "Multipliers" -> {1}]` | `-((-I + Sqrt[2])*(-I + Sqrt[3]))` | yes | 2→1 | 0.089 |
| D06 | `DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {1, 6 - 2*Sqrt[ 6] - 2*Sqrt[2] + 2*Sqrt[3]}]` | `Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.016 |

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| D07 | `DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {6 - 2*Sqrt[6] - 2*Sqrt[2] + 2*Sqrt[3]}]` | `Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.016 |
| D08 | `DenestCore[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]]` | `Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]` | yes | 3→3 | 0.114 |
| D09 | `DenestCore[2 + Sqrt[3]]` | `2 + Sqrt[3]` | yes | 1→1 | 0.001 |
| D10 | `DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {2.}]` | `1 + Sqrt[2]` | yes | 2→1 | 0.013 |
| D11 | `DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {Pi}]` | `1 + Sqrt[2]` | yes | 2→1 | 0.013 |
| D12 | `DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {1, Sqrt[5]}]` | `Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.013 |
| D13 | `DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {-1}]` | `1 + Sqrt[2]` | yes | 2→1 | 0.012 |
| D14 | `DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {I}]` | `1 + Sqrt[2]` | yes | 2→1 | 0.012 |
| D15 | `DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {1}]` | `Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.019 |
| E01 | `Strad[Sqrt[z^2]]` | `Sqrt[z^2]` | – | 1→1 | 0.017 |
| E02 | `Strad[Sqrt[t + t^2]]` | `Sqrt[t + t^2]` | – | 1→1 | 0.002 |
| E03 | `Strad[Sqrt[-x - y]]` | `Sqrt[-x - y]` | – | 1→1 | 0.002 |
| E04 | `Strad[Sqrt[a + b]]` | `Sqrt[a + b]` | – | 1→1 | 0.002 |
| E05 | `Strad[x]` | `x` | – | 0→0 | 0.000 |
| E06 | `Strad[Sqrt[x]*Sqrt[3 + 2*Sqrt[2]]]` | `(1 + Sqrt[2])*Sqrt[x]` | – | 2→1 | 0.018 |
| E07 | `Strad[f[1]]` | `f[1]` | – | 0→0 | 0.001 |
| E08 | `Strad[f[Sqrt[3 + 2*Sqrt[2]]]]` | `f[1 + Sqrt[2]]` | – | 2→1 | 0.014 |
| E09 | `Strad[ RadicalDenest‘Private‘mark[1]]` | `1` | – | 0→0 | 0.001 |

Table 19: Corrected version: factorizer and rationalizer probes (J) and miscellaneous inputs (K).

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| J01 | `Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3))]]` | `Sqrt[-(-1)^(1/3) - Sqrt[2]]` | yes | 2→2 | 0.480 |
| J02 | `Strad[Sqrt[-(Sqrt[2] + (-1)^(1/3))] + 1]` | `1 + Sqrt[-(-1)^(1/3) - Sqrt[2]]` | yes | 2→2 | 0.437 |
| J06 | `Strad[(-(Sqrt[2] + (-1)^(1/3)))^(1/3)]` | `(-(-1)^(1/3) - Sqrt[2])^(1/3)` | yes | 2→2 | 1.826 |
| J09 | `Factorc[Sqrt[-(Sqrt[2] + (-1)^(1/3))]]` | `Sqrt[-(-1)^(1/3) - Sqrt[2]]` | yes | 2→2 | 0.002 |
| J10 | `Factorc[Sqrt[-3 - 2*Sqrt[2]]]` | `I*Sqrt[3 + 2*Sqrt[2]]` | yes | 2→2 | 0.001 |
| J11 | `Factorc[Sqrt[8 + 4*Sqrt[3]]]` | `2*Sqrt[2 + Sqrt[3]]` | yes | 2→2 | 0.006 |
| J12 | `Factorc[Sqrt[z^2]]` | `Sqrt[z^2]` | – | 1→1 | 0.001 |

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| J13 | `Factorc[Sqrt[-x - y]]` | `Sqrt[-x - y]` | – | 1→1 | 0.002 |
| J14 | `Factorc[Sqrt[t + t^2]]` | `Sqrt[t*(1 + t)]` | – | 1→1 | 0.001 |
| J15 | `RationalizeDenominator[x/(1 + Sqrt[2])]` | `-x + Sqrt[2]*x` | – | 1→1 | 0.001 |
| J16 | `RationalizeDenominator[1/2 + 1/Sqrt[2]]` | `(1 + Sqrt[2])/2` | yes | 1→1 | 0.001 |
| J17 | `RationalizeDenominator[1/x]` | `x^(-1)` | – | 0→0 | 0.000 |
| K01 | `Strad[Sqrt[28^(1/3) - 3]]` | `(-1 - 2^(2/3)*7^(1/3) + 2^(1/3)*7^(2/3))/3` | yes | 2→1 | 0.121 |
| K02 | `Strad[(1 + 2^(1/3))^3]` | `(1 + 2^(1/3))^3` | yes | 1→1 | 0.007 |
| K04 | `Strad[Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]], True], True]` | `Sqrt[5] + Sqrt[11 - 2*Sqrt[29]]` | yes | 2→2 | 1.762 |
| K09 | `Strad[Sqrt[5 + 2*Sqrt[6]]/Sqrt[3 + 2*Sqrt[2]]]` | `2 - Sqrt[2] - Sqrt[3] + Sqrt[6]` | yes | 2→1 | 0.284 |
| K10 | `Strad[Exp[I*(Pi/7)]*Sqrt[3 + 2*Sqrt[2]]]` | `(-1)^(1/7)*(1 + Sqrt[2])` | yes | 2→1 | 0.107 |
| K11 | `Strad[Sqrt[3 + 2*Sqrt[2]] + Pi]` | `1 + Sqrt[2] + Pi` | yes | 2→1 | 0.035 |
| K12 | `Strad[Sqrt[3.  + 2*Sqrt[2]]]` | `2.414213562373095` | yes | 0→0 | 0.000 |
| K14 | `Strad[{Sqrt[3 + 2*Sqrt[2]], Sqrt[5 + 2*Sqrt[6]]}]` | `{1 + Sqrt[2], Sqrt[2] + Sqrt[3]}` | – | 2→1 | 0.103 |
| K15 | `Strad[Sqrt[3 + 2*Sqrt[2]] == 1 + Sqrt[2]]` | `True` | – | 0→0 | 0.002 |
| K17 | `Strad[Sqrt[(3 + 2*Sqrt[2])/(5 + 2*Sqrt[6])]]` | `-2 - Sqrt[2] + Sqrt[3] + Sqrt[6]` | yes | 2→1 | 0.206 |
| K19 | `Strad[Sqrt[Sqrt[2] + 1]^3]` | `(1 + Sqrt[2])^(3/2)` | yes | 2→2 | 0.065 |
| K20 | `Strad[Sqrt[2 + Sqrt[3]]*Sqrt[3 + 2*Sqrt[2]]]` | `((2 + Sqrt[2])*(1 + Sqrt[3]))/2` | yes | 2→1 | 0.148 |
| K21 | `Strad[Sqrt[3 + 2*Sqrt[2]], "Verbose" -> False, "MaxTrials" -> 1]` | `1 + Sqrt[2]` | yes | 2→1 | 0.035 |
| K22 | `Strad[Sqrt[5 + 2*Sqrt[6]], "Factor" -> False]` | `Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.041 |

Table 20: Corrected version: higher-order roots (L) and roots of negative bases (Q).

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| L01 | `Strad[(239 + 169*Sqrt[2])^(1/7)]` | `1 + Sqrt[2]` | yes | 2→1 | 0.067 |
| L02 | `Strad[(2 + Sqrt[2])^(1/5)]` | `(2 + Sqrt[2])^(1/5)` | yes | 2→2 | 0.430 |
| L03 | `Strad[(1 + Sqrt[2])^(1/7)]` | `(1 + Sqrt[2])^(1/7)` | yes | 2→2 | 0.773 |
| L04 | `Strad[Sqrt[17 + 2*Sqrt[6] + 2*Sqrt[10] + 2*Sqrt[14] + 2*Sqrt[15] + 2*Sqrt[21] + 2*Sqrt[35]]]` | `Sqrt[2] + Sqrt[3] + Sqrt[5] + Sqrt[7]` | yes | 2→1 | 2.916 |

| ID | Input | Output | Equal | Depth | Time (s) |
|---|---|---|---|---|---|
| L05 | `Strad[Expand[(1 + Sqrt[2] + Sqrt[3])^3]^(1/3)]` | `1 + Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.118 |
| L06 | `Strad[Expand[(Sqrt[2] + Sqrt[3])^4]^(1/4)]` | `Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.057 |
| L07 | `Strad[Sqrt[1000003 + 2*Sqrt[999983]]]` | `Sqrt[1000003 + 2*Sqrt[999983]]` | yes | 2→2 | 0.212 |
| L08 | `Strad[Sqrt[2 + Sqrt[3] + Sqrt[5]]]` | `Sqrt[2 + Sqrt[3] + Sqrt[5]]` | yes | 2→2 | 0.376 |
| L09 | `Strad[(5 + 2*Sqrt[6])^(1/6)]` | `(5 + 2*Sqrt[6])^(1/6)` | yes | 2→2 | 1.554 |
| L10 | `Strad[(11*Sqrt[2] + 9*Sqrt[3])^(1/3)]` | `Sqrt[2] + Sqrt[3]` | yes | 2→1 | 0.054 |
| L11 | `Strad[Expand[(2^(1/3) + 3^(1/3))^3]^(1/3)]` | `2^(1/3) + 3^(1/3)` | yes | 2→1 | 0.266 |
| L12 | `Strad[Sqrt[Expand[(1 + 2^(1/3) + 3^(1/3))^2]]]` | `1 + 2^(1/3) + 3^(1/3)` | yes | 2→1 | 0.058 |
| L13 | `Strad[Expand[(1 + 2^(1/5))^5]^(1/5)]` | `1 + 2^(1/5)` | yes | 2→1 | 0.055 |
| L14 | `Strad[Sqrt[5 + 2*Sqrt[6]]^(1/5)]` | `(5 + 2*Sqrt[6])^(1/10)` | yes | 2→2 | 6.605 |
| Q01 | `Strad[Expand[(-1 - Sqrt[2])^3]^(1/3)]` | `((1 + Sqrt[2])*(1 + I*Sqrt[3]))/2` | yes | 2→1 | 0.401 |
| Q02 | `Strad[Expand[(1 - Sqrt[2])^3]^(1/3)]` | `((-1 + Sqrt[2])*(1 + I*Sqrt[3]))/2` | yes | 2→1 | 0.366 |
| Q07 | `Strad[Expand[(2 - 3*Sqrt[2])^3]^(1/3)]` | `((-I)*(-3 + Sqrt[2])*(-I + Sqrt[3]))/Sqrt[2]` | yes | 2→1 | 0.475 |
| Q08 | `Strad[(-7 - 5*Sqrt[2])^(1/3)]` | `((1 + Sqrt[2])*(1 + I*Sqrt[3]))/2` | yes | 2→1 | 0.212 |

### 5.7 Probing the corrected version for misses

After the battery above, four further scripts (`harness/probe\_gaps\_1..4.wl`, transcripts in `logs/`) ran 136 denestable or control inputs drawn from the literature guide and from constructed identities through `Strad` with a 400 s wall limit. No wrong value was returned. Of the 111 denestable inputs not already evaluated by the kernel itself, 96 were denested to the expected depth: multiquadratic sums with up to four independent square roots, depth-3 inputs, the Ramanujan and Honsbeek identities, mixed-index sums such as $\sqrt{2+2\sqrt{2}\sqrt[3]{3}+\sqrt[3]{9}}=\sqrt{2}+\sqrt[3]{3}$, complex radicands, and cube, fifth and seventh roots of positive elements of quadratic fields. The 15 misses fall into five families (Table 21); the file `src/corrected/KNOWN\_GAPS.md` gives every input, the verbose diagnosis and a proposed fix for each.

## 6 Unified catalogue of defects and shortcomings

Each entry gives the severity, the source lines, the reviews that report it, the experimental evidence, and the repair adopted in `StradFixed.wl` (Section 7). Severity: **critical** = wrong value can be returned; **high** = crash, garbage output, or uncontrolled resource use; **medium** = wrong or misleading behaviour without a wrong value; **low** = maintenance.

## 6.1 Wrong values

### U1 The candidate filter certifies against $(\beta^q)^{1/q}$ instead of $\beta$

*Severity:* critical. *Source:* lines 94, 125–127. *Reviews:* R1 F01, R2 B1, R3 F1.

*Evidence.* D01, D04, D05 (engine), C02, C03 (wrapper); fuzz F5 39/40 and F6 17/40 wrong. The wrapper's grouping rules and its acceptance of exponents $p/q$ with $p > 1$ deliver exactly the values outside the principal sector of Proposition 3.3. The reviews' hedge that a leading minus sign is kept outside the marker is confirmed (C01 is right). *Repair.* `tryMultiplier` receives `problem` and selects with `CertifiedEqualQ[##, problem]`; the orbit of Proposition 3.2 is unchanged.

### U2 The custom factorizer applies power identities without branch conditions

*Severity:* critical. *Source:* lines 527–531, 571–583. *Reviews:* R1 F02/F03, R2 B2, R3 F2.

*Evidence.* Symbolic: E01 $\sqrt{z^2} \to z$, E03 $\sqrt{-x-y} \to i\sqrt{x+y}$, H18 $\sqrt{x^4} \to x^2$, H19 $(x^2)^{1/3} \to x^{2/3}$. Numeric, end to end: J01/J02 $\sqrt{-(\sqrt{2}+(-1)^{1/3})} \to i\sqrt{(-1)^{1/3}+\sqrt{2}}$ (the negative of the input), J06 the cube-root analogue. R1's symbolic probe `Sqrt[t + t^2]` does *not* fail (H02, and the passing test in its own suite). *Repair.* Proposition 3.5: the exponent merge is applied only when the outer exponent is an integer or the inner exponent is real in $(-1, 1]$; common factors are pulled out of a sum under a fractional power only when the power is an integer, the extracted factor is a positive real, or the remaining sum is a positive real. Factoring is applied only to exact numeric subexpressions and its output is certified against its input.

## 6.2 Unsafe acceptance and validation

### U3 `PossibleZeroQ` with default method is the only certificate

*Severity:* high. *Source:* lines 125–127. *Reviews:* R1 F04, R2 §8.3, R3 F6.

*Evidence.* In 20 of 31 classical rows and 272 of 340 random inputs the kernel reports `PossibleZeroQ::ztest1`: it could not decide and assumed zero. Because the candidates are exact algebraic numbers and the correct one is present in the orbit, this "assumption" happened to be right in every observed case, and S04–S06 show that the default method does reject close non-zero differences. The defect is a missing guarantee, not an observed false positive. *Repair.* `CertifiedEqualQ`: 30-digit numeric rejection, then `RootReduce[a - b] === 0`, then `PossibleZeroQ[..., Method -> "ExactAlgebraics"]`, each under a time limit; undecided means rejected. No message is emitted.

### U4 Zero, inexact or non-algebraic multipliers and empty candidate lists are not handled

*Severity:* high. *Source:* lines 111–134, 214–215. *Reviews:* R1 F05, R2 B5/B10, R3 F5.

*Evidence.* D02 returns `{}[[1]]` after printing "success"; D10 returns `Outer[][[1]]`; both leave `Power::infy` and `Part::partw` messages. *Repair.* Multipliers must be exact, algebraic and nonzero; polynomial results are type-checked; an empty certified list is an ordinary "no candidate" outcome.

#### U15 The input domain is not enforced

*Severity:* high. *Source:* lines 5, 79, 454. *Reviews:* R1 F10, R2 B10, R3 F9.

*Evidence.* K14: a list of two radicals becomes $(3+2\sqrt{2})^{(\sqrt{2}+\sqrt{3})/2}$ because the nesting helpers and the factorizer encode structure as lists. K12: an inexact input is returned as a machine number. B07: a `Root` object becomes a nested radical. E08: a user function around a radical triggers seven messages. *Repair.* Lists are mapped over; inexact input is returned unchanged; `Root` input is converted with `ToRadicals` first and then denested; only exact numeric algebraic subexpressions are factored and marked.

### 6.3 Search control

#### U5 The prime comparator compares a pair with itself

*Severity:* high. *Source:* lines 181. *Reviews:* R1 F06, R2 B8, R3 F3.

*Evidence.* S01: `Sort` with the predicate `##1[[1]] <= ##[[2]] &` returns the list *reversed* (the predicate is `False` for every pair of plain primes and the merge sort then reverses). Consequences (N01): the ratio-pruning loop of lines 184–189 never runs, and the prefix of lines 190–194 keeps the largest primes. *Repair.* `SortBy[..., First]`.

#### U6 The prefix length uses the pre-deletion length

*Severity:* high (latent). *Source:* lines 190–194. *Reviews:* R3 F4.

*Evidence.* S02 and N02: with the comparator repaired, the fixture $\{2, 3, 5, 7, 11, 2003\}$ triggers `Part::take`. In the original program the zeroing loop never runs (U5), so the defect is unreachable; it becomes live as soon as U5 is fixed in isolation. *Repair.* Delete first, then take `Min[count, Length]`.

#### U7 The multiplier "cap" is not a cap; `Ceiling[Log]` misfires

*Severity:* high. *Source:* lines 162–199. *Reviews:* R1 F07, R2 B6, R3 F7.

*Evidence.* Proposition 3.7; N05/N06 materialize 1024 and 100 000 divisors; the largest batch observed in practice was 63 (L04). For $q = 10$ the exact quotient `Log[1000]/Log[10]` is not decided by `Ceiling` (R01–R04, L14), so `Max[5, ...]` silently falls back to 5. *Repair.* $t = \max\{t : q^t - 1 \leq \text{cap}\}$ by integer arithmetic; the post-first-pass product batch is truncated to the cap; a trial budget and a time budget bound the whole search.

#### U12 The equal-degree gate skips a linear GCD

*Severity:* medium. *Source:* lines 115. *Reviews:* R2 H1.

*Evidence.* D06 versus D07. *Repair.* The GCD is attempted when $d_m \leq d_{\text{best}}$.

#### U13 `List @@ Expand[...]` assumes a sum

*Severity:* low. *Source:* lines 224–229. *Reviews:* R1 §5.1, R2 H4, R3 §6.1.

*Evidence.* Not reachable with exact numeric radicands: `Expand` of a power of a nested radical is a `Plus`, and monomial radicands are split by the kernel's automatic evaluation before they reach the engine (`Sqrt[3 Sqrt[2]]` evaluates to `2^(1/4) Sqrt[3]`). *Repair.* An explicit `summands` helper.

### U14 Composite cofactors of the discriminant are dropped

*Severity:* low. *Source:* lines 173–181. *Reviews:* R1 §4.4, R2 H2, R3 §6.3.

*Evidence.* Never triggered in the experiments (no "composite factor" diagnostic was printed). *Repair.* Kept, but logged.

### U16 No global budget and no visited set

*Severity:* medium. *Source:* lines 251–432. *Reviews:* R1 F12, R2 B7, R3 §9.3.

*Evidence.* Duplicate multipliers across batches are visible in the traces (D08 tries 2 and $2\sqrt{3+2\sqrt{2}}$, then 17, 34, $17\sqrt{\cdot}$, $34\sqrt{\cdot}$); no runaway was observed on the tested inputs. *Repair.* `"MaxTrials"`, `"TimeBudget"`, and a visited association keyed by the multiplier.

### U22 Single-level mode cannot exploit inner denestings

*Severity:* medium. *Source:* lines 7–10, 58–61. *Reviews:* all, in passing.

*Evidence.* A29 (single level) leaves $\sqrt{3+2\sqrt{3+2\sqrt{2}}}$ alone after 12 trials; A28 (all levels) finds $\sqrt{5+2\sqrt{2}}$. *Repair.* The default remains single level for compatibility, but a partial solution of depth $\geq 2$ is passed through one further all-level pass.

### U23 Candidates contain denestable inner surds

*Severity:* medium. *Source:* lines 121–134. *Reviews:* –.

*Evidence.* Q01–Q08: the roots of the quadratic GCD contain $\sqrt{9-6\sqrt{2}} = \sqrt{6}-\sqrt{3}$ and `Simplify` does not remove it, so 17 of 30 negative-base cube roots keep their depth. *Repair.* Candidate polishing applies Proposition 3.6 to inner square roots, then `Simplify`; all 30 are denested.

### U24 "Forced" multipliers are only initial multipliers

*Severity:* low. *Source:* lines 214–215, 338–378. *Reviews:* all.

*Evidence.* D12: with $\{1, \sqrt{5}\}$ the degree-8 trial counts as progress and a discriminant batch finds 2. *Repair.* Documented as the `"Multipliers"` option with the same semantics.

### U25 The search is heuristic and incomplete

*Severity:* medium. *Source:* lines 159–249. *Reviews:* R1 F11, R2 H3, R3 F11.

*Evidence.* No false "denesting exists" claims were observed; A13 and the F3 controls are correctly unchanged. No completeness certificate is possible for this design. *Repair.* None; see Section 8.

## 6.4 Output quality

### U9 "Success" is not measured against an objective

*Severity:* medium. *Source:* lines 120–134, 309–324, 435–438. *Reviews:* R1 F09, R2 H5, R3 F8.

*Evidence.* D07 returns $(1+\sqrt{2}+\sqrt{3})/\sqrt{2(3-\sqrt{2}+\sqrt{3}-\sqrt{6})}$ for $\sqrt{5+2\sqrt{6}}$; A30/A31 and L04 return correct but huge quotients; B01 rewrites $\sqrt{2+\sqrt{2}}$ as $2^{1/4}\sqrt{1+\sqrt{2}}$. *Repair.* The cost $\mathrm{cost}(E) = (\#\texttt{Root}, \text{rdepth}, \#\text{radical nodes}, \#\text{atoms}, \texttt{LeafCount})$ is compared lexicographically; a result is returned only if it is strictly cheaper than the input. Candidates are polished by `Simplify`, rationalization of the denominator (which turns the L04 quotient into $\sqrt{2}+\sqrt{3}+\sqrt{5}+\sqrt{7}$), and inner-surd denesting.

### U20 The final `Simplify` re-nests `Root` objects and rewrites unchanged input

*Severity:* medium. *Source:* lines 59–61, 73. *Reviews:* R3 in passing.

*Evidence.* B07, B01, and the 8 "equal, rewritten" F3 outputs. *Repair.* `Simplify` is one of several candidate variants subject to the cost gate, and `Root` input is converted *before* denesting.

### U27 Quotient candidates are not rationalized

*Severity:* low. *Source:* lines 121–134. *Reviews:* –.

*Evidence.* L04; the program's own `RationalizeDenominator` followed by `Simplify` produces the intended answer (S01 in Table 13). *Repair.* Part of candidate polishing.

## 6.5 Interfaces and session hygiene

### U8 Loading the file unprotects `Print`

*Severity:* medium. *Source:* lines 2. *Reviews:* R1 F08, R2 B9, R3 F10.

*Evidence.* `Attributes[Print]` is `{Protected}` before and `{}` after `Get`. *Repair.* Verbose logging through an option.

### U10 The marker `f` collides with user symbols

*Severity:* high. *Source:* lines 6, 12, 55. *Reviews:* R2 B3.

*Evidence.* M01/E07 (`f[1]` becomes `1`), E08. *Repair.* A private-context marker symbol. E09 shows that even the private marker is handled gracefully.

### U11 `RationalizeDenominator` substitutes `x -> 0` into the numerator

*Severity:* high. *Source:* lines 445–450. *Reviews:* R2 B4.

*Evidence.* R05 returns 0 for `x/(1 + Sqrt[2])`. *Repair.* `Module`-local variable; non-algebraic denominators are returned unchanged (J17).

#### U19 `Strad` suppresses `Print` but not messages

*Severity:* medium. *Source:* lines 3. *Reviews:* –.

*Evidence.* Ordinary successful calls show `N::meprec` and `PossibleZeroQ::ztest1`; invalid input shows `Power::infy`, `Part::partw`, and others. *Repair.* No message-producing operation remains on the normal path; failures are caught and turned into "no candidate".

#### U17 Fragile positional state; `Throw[, "solution"]`

*Severity:* low. *Source:* lines 253–267, 292, 320. *Reviews:* R1 F13, R2 M1, R3 §9.5.

*Evidence.* Loading emits `Syntax::com` for lines 320. *Repair.* `Throw[Null, "solution"]`; otherwise the record layout is kept for fidelity, with type checks where records are consumed.

#### U18 Dead settings and unused variables

*Severity:* low. *Source:* lines 95–103, 163, 400. *Reviews:* R1 F14, R2 M1, R3 F12.

`extrapower`, `maxprimes`, `best` (in the extra block), `expected`, `done`. *Repair.* Removed.

#### U21 `combine` does not clean a trailing unit entry

*Severity:* low. *Source:* lines 558–568. *Reviews:* R3 §9.5.

*Repair.* Loop bound `j <= Length`.

#### U26 All-level mode marks every radical, including $\sqrt{29}$

*Severity:* low. *Source:* lines 9, 11. *Reviews:* –.

*Evidence.* The A30 trace shows `f[Sqrt[29]]` as a "problem". Harmless: the engine returns it at its gate.

### 6.6 What is *not* a defect

The sign-flip loop with its accumulator (lines 30–37) preserves the product; one-argument `MinimalPolynomial` returning a pure function is valid; non-strict comparators are permitted by `Sort`; `Extension -> Automatic` handles nested radicands (D08); R3's level-0 `MemberQ` remark does not apply to exact numeric input; and the rationalizer's formula is correct including the non-monic case (R01–R10). The engine's search order, though heuristic, found every classical denesting we tried, usually with the first or second discriminant batch.

## 7 The corrected version

`src/corrected/StradFixed.wl` (Appendix B, about 480 lines) is a package in the context `RadicalDenest‘`. It deliberately keeps the original architecture (wrapper, markers, grouping, multiplier search, history and stack, discriminant batches, the post-first-pass product batch) so that the search behaviour stays recognizable, and changes only what the catalogue requires. The design rule is the one all three reviews recommend: *heuristics propose, exact algebra disposes.*

## 7.1 Interface

```
Strad[expr]                          (* single level, as before *)
Strad[expr, True]                    (* all levels, as before *)
Strad[expr, "AllLevels" -> True, "Verbose" -> True]
DenestCore[problem, "Multipliers" -> {2}]   (* the engine, formerly denest11 *)
```

Options: `"AllLevels"` (`False`), `"Verbose"` (`False`), `"Multipliers"` (`Automatic` or an initial list), `"MaxTrials"` (400), `"TimeBudget"` (120 s), `"MultiplierCap"` (1000), `"CertifyTime"` (20 s per exact decision), `"Solver"` (`Automatic` or a function replacing the engine), `"Factor"` (`True`). Lists are mapped over. Public helpers: `RationalizeDenominator`, `Factorc`, `RadicalDepth`, `RadicalCost`, `ExactAlgebraicQ`, `CertifiedEqualQ`.

## 7.2 Change list

1. **Certification** (U1, U3). `CertifiedEqualQ[a, b]` returns `True` only if $a - b$ is proved zero: exact inputs, a 30-digit numeric rejection, `RootReduce`, then `PossibleZeroQ` with `Method -> "ExactAlgebraics"`; every step is time limited and "undecided" is "not equal". The engine's filter is `Select[possibilities, CertifiedEqualQ[##, problem] &]`. The wrapper certifies its whole result against the whole input whenever the input is an exact number.
2. **Branch-safe factorizer** (U2). `fullfactor` merges $\{a^b, c\}$ into $\{a, bc\}$ only if `powerMergeSafeQ[b, c]` (Proposition 3.5(1)); `combine` extracts common factors from a sum under exponent $p$ only if $p \in \mathbb{Z}$, the extracted product is a positive real, or the remaining sum is a positive real (Proposition 3.5(2)); `Factorc` certifies its own output; the wrapper factors only maximal exact numeric non-atomic subexpressions. The useful rewrites survive: J10 $\sqrt{-3-2\sqrt{2}} \to i\sqrt{3+2\sqrt{2}}$ and J11 $\sqrt{8+4\sqrt{3}} \to 2\sqrt{2+\sqrt{3}}$.
3. **Validation** (U4, U15). Exact algebraic nonzero multipliers; integer root order $\geq 2$; polynomial-type checks after `MinimalPolynomial`, `PolynomialGCD` and `Solve`; empty candidate sets are ordinary; inexact input is returned; `Root` input goes through `ToRadicals` first; lists are mapped.
4. **Search control** (U5, U6, U7, U12, U16). `SortBy[..., First]`; delete-then-take; the cap $q^t - 1 \leq$ cap in integer arithmetic; the GCD attempted for $d_m \leq d_{\text{best}}$; a visited set; `"MaxTrials"` and `"TimeBudget"` checked before every trial; the product batch after the first pass truncated to the cap; `Throw[Null, ...]`.
5. **Objective** (U9, U20, U27). `RadicalCost` and `cheaperQ`; the engine keeps the cheapest certified candidate and only replaces a solution by a cheaper one; the wrapper returns the cheapest of {input, result, `Simplify[result]`, inner-surd variant} that is certified equal to the input, and never anything more expensive than the input.
6. **Candidate polishing** (U23, U27). Each certified candidate is offered in the variants {as is, `Simplify`, rationalized denominator, inner surds denested by Proposition 3.6, both}, and the cheapest is kept.
7. **Quadratic-surd fast path** (R3's proposal). A problem of the form $\sqrt{a + b\sqrt{c}}$ with rational $a, b, c$ is tried with Proposition 3.6 before any minimal polynomial is computed; the result is certified anyway.

8. **One recursive pass** (U22). A partial solution of depth $\geq 2$ is sent once through an all-level pass (recursion depth at most 2), so $(49 + 20\sqrt{6})^{1/4} \to \sqrt{5 + 2\sqrt{6}} \to \sqrt{2} + \sqrt{3}$.
9. **Hygiene** (U8, U10, U11, U17, U18, U19, U21). Package context; private marker; `Module` variables; no `Unprotect`; `log[...]` instead of `Print`; dead code removed; the trailing unit entry cleaned.

### 7.3 What was deliberately kept

The initial-multiplier heuristic, the degree-based "progress" logic, the history ordering, the stack discipline (split-and-insert for partial batches, deferred insertion for full-set batches), the discriminant batches with prime-gap trimming, the product batch after the first pass, and the early stopping rule are all retained. They are heuristics that the experiments show to be effective on the classical corpus, and replacing them was not the goal of a *corrected* version. Their remaining weaknesses are listed in Section 8.

### 7.4 Guarantees and their limits

For an exact algebraic input $E$ the corrected version guarantees: (i) the result is certified equal to $E$ by `RootReduce` or exact `PossibleZeroQ`; (ii) the result is strictly cheaper than $E$ under `RadicalCost`, or is $E$ itself; (iii) no kernel message is produced on the normal path; (iv) the search stops after `"MaxTrials"` trials or `"TimeBudget"` seconds. For an input with symbolic parts, (i) holds for every replaced numeric subexpression individually and the symbolic host is left untouched. There is no completeness guarantee: an unchanged result means that the bounded search found nothing cheaper, not that nothing exists. Certification can time out on very large expressions, in which case candidates are rejected and the input is returned; this is the conservative direction.

### 7.5 Cost

The corrected version is two to three times slower than the original on the small inputs of the battery (Table 16 versus Table 4): almost all of the difference is `RootReduce` inside `CertifiedEqualQ`, called for the Factorc check, for each candidate that passes the numeric prefilter, and once more for the whole result. The slowest probe, $\sqrt{5 + 2\sqrt{6}}^{1/5}$, takes 7 s (20 s in the original).

## 8 Future improvements

The corrected version makes the program *safe*; it does not make it *complete* or *fast*. Section 5.7 exhibits fifteen denestable inputs it leaves unchanged; items 1, 3 and 5 below address them directly. The following improvements are listed in the order in which we would undertake them.

### 8.1 Algorithmic

1. **Replace degree heuristics by field computations.** The engine guesses multipliers and asks a degree oracle whether it guessed well. For a problem $\beta$ with $\rho = \beta^q$, the question "does $X^q - \rho$ have a linear factor over $K = \mathbb{Q}(\rho)$?" is answered directly by `Factor[x^q - rho,`

`Extension -> Automatic]`; a linear factor is a denesting into $K$ with no multiplier at all, and the trial $m = 1$ is then redundant. More generally, for $\beta$ of degree $n$ over $\mathbb{Q}$ the candidate fields for a depth reduction are the subfields of the Galois closure; Landau's algorithm for nested square roots and Blömer's algorithms for real radicals and for roots of unity reduce the problem to computing a specific extension (e.g. adjoining $\sqrt[q]{d}$ for a *single* well-chosen $d$ derived from the discriminant or from the norm of $\rho$) and testing membership. The discriminant-prime batches of `newmultipliers` are a heuristic approximation of this; the exact version would try $m = \pm d^e$ for the $q$-th-power-free divisors $d$ of the norm $N_{K/\mathbb{Q}}(\rho)$ and its conjugates, which is a finite and provably sufficient list for Kummer-type denestings ($\sqrt[q]{\rho} \in K(\sqrt[q]{m})$ for some $m \in K$).

2. **A complete quadratic-surd layer.** Proposition 3.6 covers the case $a^2 - b^2c = \delta^2$. The other half of Landau's theorem, $c(a^2 - b^2c) = \delta^2$, yields denestings with a fourth root ($\sqrt{a + b\sqrt{c}} = c^{1/4}\sqrt{\cdots}$ forms); and nested square roots over a real quadratic field $\mathbb{Q}(\sqrt{d})$ admit an exact decision procedure (Borodin–Fagin–Hopcroft–Tompa) that would find A30 in one step instead of a two-level multiplier search. Both are cheap to add in front of the engine.

3. **Denest sums and products jointly.** The wrapper treats $\sqrt{2+\sqrt{3}} + \sqrt{2-\sqrt{3}}$ (A23) as two problems and relies on `Simplify` to combine the answers. A joint treatment ($\beta = \beta_1 + \beta_2$ with $\beta^2 \in \mathbb{Q}(\sqrt{3})$) is the Borodin–Fagin–Hopcroft–Tompa setting and would also cover $\sqrt{a+\sqrt{b}} + \sqrt{a-\sqrt{b}}$ patterns that the current design cannot see.

4. **Root objects as first-class citizens.** The corrected version converts `Root` input to radicals and forbids `Root` in the output by its cost. A "canonical" mode could instead accept `Root` output (`RootReduce` is the canonical form) and use `ToRadicals` only when it yields a radical-only expression of lower depth.

5. **Roots of unity and complex bases.** Cube roots of negative reals denest to $e^{i\pi/3}(\cdots)$; the corrected version finds these but prints them as $(1 + i\sqrt{3})/2 \cdot (\cdots)$. A normal form that writes $(-1)^{p/q}$ factors explicitly and a fast path for $\rho = -|\rho|$ would give shorter answers in one step.

## 8.2 Search engineering

1. **Lazy, prioritized generation.** Batches are still materialized as lists (bounded by the cap). A priority queue of *generators* (exponent vectors over the selected primes, ordered by a cost such as $\sum e_i \log p_i$) would remove the cap altogether and make the trial order independent of `Divisors`'s ordering.

2. **Caching.** `MinimalPolynomial` of $(m\rho)^{1/q}$ is the dominant cost per trial and is recomputed for algebraically equal multipliers written differently. Key the visited set by `RootReduce[m]` (with a small time limit) instead of by syntax.

3. **Cheaper certification.** `RootReduce` is called on differences that are exactly zero, which is its worst case. A modular or interval prefilter with a proven bound (e.g. compare at 200 digits and only then call `RootReduce`), or certification by `MinimalPolynomial[candidate] == MinimalPolynomial[problem]` plus a numeric separation argument between conjugates, would cut the factor of two to three observed in Section 5.6.

4. **Structured results.** The engine should return an association (status, best certified value, cost, trials used, budget exhausted?, multipliers tried) so that callers can distinguish "nothing

found" from "budget exhausted" and "certification timed out". The current corrected version returns only the expression for compatibility.

5. **Records instead of positional lists.** The history and stack layout of the original is preserved; replacing it by associations with explicit keys (`"Multiplier"`, `"Degree"`, `"Batch"`, `"Deferred"`) would let the invariants be checked and the scheduling be unit-tested with a mock trial function.

6. **Termination.** The outer loop compares its index with a growing stack. The budget makes termination certain, but a monotone measure (each inserted batch strictly decreases the best degree or is a product batch inserted at most once) would make it provable without a budget.

### 8.3 Testing and quality

1. **Property-based tests from constructed answers.** Every family of Section 5.3 is of the form "build $\beta$, hand over $\beta^q$ raised to $1/q$"; the same generator can produce nested answers ($\beta = \sqrt{5} + \sqrt{11 - 2\sqrt{29}}$), roots of unity, and products, and the oracle is exact equality plus a depth target.

2. **Regression suite.** The harness of Appendix C and the reviews' three `.wlt` files should be merged into one `TestReport` suite run on both versions; all expected values used here were verified with `RootReduce` in the kernel.

3. **Timing corpus.** The battery is small-input only. A corpus of degree-16 to degree-64 problems (four to six independent square roots, cube roots of sums of three cube roots) is needed before any claim about performance; the original's 20 s tenth root suggests where the cliff is.

4. **Output normal form.** Choose and document one: the corrected version prefers fewer radical nodes and then fewer atoms, which yields $\sqrt{3/2} + 1/\sqrt{2}$ for A03 where the original's `Simplify` gave $(1 + \sqrt{3})/\sqrt{2}$. Both are depth 1; a documented tie-break (e.g. rationalized denominators, integer radicands) would remove the surprise.

## 9 Conclusion

The three reviews were right about the essentials and, being static, could not see the rest. Executed in Wolfram 15.0.1, `Strad.wl` denests every classical identity in our corpus quickly and correctly, and its two mathematical ideas, the multiplier-driven polynomial GCD over an algebraic extension and the minimal-polynomial rationalizer, are sound. It also returns wrong values on a large, natural class of inputs: 56 of 340 random inputs, concentrated entirely in complex products and non-principal powers, plus a purely numeric case caused by its own factorizer. Its acceptance test rests on `PossibleZeroQ` assuming zero when it cannot decide; its prime comparator reverses the prime list and thereby disables its own pruning while hiding a second bug; it mishandles lists, inexact numbers and `Root` objects; and several of its "successes" are larger than the input.

The unified catalogue contains 27 items. Sixteen were reported by at least one review and are confirmed here; five were reported and are refined (one is unreachable, one is moot, one probe was mis-chosen, two are practically inert); six are new. The corrected package removes every wrong value observed and every message, adds budgets and an explicit improvement criterion, and finds several denestings the original misses, at a cost of two to three in running time. Everything reported is reproducible from the harness, logs and scripts shipped alongside this document.

| Unified | Short description | R1 | R2 | R3 | Status |
|---|---|---|---|---|---|
| U1 | Candidates certified against $(\beta^q)^{1/q}$, not $\beta$ | F01 | B1 | F1 | confirmed, quantified |
| U2 | Branch-unsafe power rules in the factorizer | F02, F03 | B2 | F2 | confirmed; new numeric case |
| U3 | `PossibleZeroQ` as certificate | F04 | ○ | F6 | confirmed: "assuming zero" in practice |
| U4 | Zero/invalid multipliers, `{}[[1]]` | F05 | B5, B10 | F5 | confirmed |
| U5 | Comparator `##1[[1]] <= ##[[2]]` | F06 | B8 | F3 | confirmed; reverses the list |
| U6 | Stale length in the prime slice | – | – | F4 | confirmed snippet; *unreachable* in original |
| U7 | Cap is not a cap; `Ceiling[Log]` | F07 | B6 | F7 | confirmed; new `Ceiling::meprec` |
| U8 | `Unprotect[Print]` at load | F08 | B9 | F10 | confirmed |
| U9 | "Success" is not an improvement | F09 | H5 | F8 | confirmed; several ugly outputs |
| U10 | Marker `f` collides with user data | – | B3 | – | confirmed |
| U11 | Rationalizer's `x -> 0` erases a numerator | – | B4 | – | confirmed |
| U12 | Equal-degree gate skips a linear GCD | – | H1 | ○ | confirmed |
| U13 | `List @@ Expand` head assumption | ○ | H4 | ○ | not reachable with exact input |
| U14 | Composite discriminant cofactors dropped | ○ | H2 | ○ | theoretical; not observed |
| U15 | Input domain unenforced (lists, inexact, `Root`) | F10 | B10 | F9 | confirmed: list input is nonsense |
| U16 | No budget, no visited set | F12 | B7 | ○ | confirmed duplicates; no blow-up seen |
| U17 | Fragile positional state; `Throw[,]` | F13 | M1 | ○ | new: load-time `Syntax::com` |
| U18 | Dead settings | F14 | M1 | F12 | confirmed by inspection |
| U19 | Messages leak through `Strad` | – | – | – | new |
| U20 | Final `Simplify` re-nests `Root`, rewrites non-denested input | – | – | ○ | new |
| U21 | `combine` skips the last unit entry | – | – | F11 | confirmed by inspection |
| U22 | Single-level mode cannot use inner denestings | ○ | ○ | ○ | confirmed (A29 vs A28) |
| U23 | Candidates contain denestable inner surds | – | – | – | new: 17/30 cube roots not denested |
| U24 | "Forced" multipliers are only initial | ○ | ○ | ○ | confirmed (D12) |
| U25 | Search incompleteness in general | F11 | H3 | F11 | theoretical; A13 correctly unchanged |
| U26 | All-level mode marks trivial radicals | – | – | – | new, harmless |
| U27 | Poor final form (quotients, unrationalized) | ○ | ○ | ○ | new: 4-term and BFHT outputs |



Table 3: Coverage of the unified findings by the three reviews.

| | | Original | | | | Corrected | | |
|---|---|---|---|---|---|---|---|---|
| Family | $n$ | denested | equal, rewritten | unchanged | **wrong** | denested | unchanged | **wrong** |
| F1 $\sqrt{(p+q\sqrt{c})^2}$ | 40 | 40 | 0 | 0 | 0 | 40 | 0 | 0 |
| F2 $\left((p+q\sqrt{c})^3\right)^{1/3}$ | 30 | 13 | 17 | 0 | 0 | 30 | 0 | 0 |
| F3 $\sqrt{a+b\sqrt{c}}$, random | 40 | 6 | 8 | 26 | 0 | 6 | 34 | 0 |
| F4 $\sqrt{(p+qi\sqrt{c})^2}$ | 40 | 40 | 0 | 0 | 0 | 40 | 0 | 0 |
| F5 $\left((p+qi\sqrt{c})^2\right)^{3/2}$ | 40 | 1 | 0 | 0 | **39** | 40 | 0 | 0 |
| F6 $\sqrt{u^2}\sqrt{v^2}$, complex | 40 | 23 | 0 | 0 | **17** | 40 | 0 | 0 |
| F7 $\left((p+q\sqrt{c})^4\right)^{1/4}$ | 25 | 17 | 8 | 0 | 0 | 25 | 0 | 0 |
| F8 $\sqrt{(p+q\sqrt{c}+r\sqrt{d})^2}$ | 25 | 25 | 0 | 0 | 0 | 25 | 0 | 0 |
| F9 $\left((p+q\sqrt{c})^2\right)^{3/2}$, real | 30 | 30 | 0 | 0 | 0 | 30 | 0 | 0 |
| F10 $\sqrt{-u^2}\sqrt{-v^2}$, real | 30 | 30 | 0 | 0 | 0 | 30 | 0 | 0 |
| Total | 340 | 225 | 33 | 26 | **56** | 306 | 34 | **0** |

Table 14: Randomized families. The original program emitted `PossibleZeroQ::ztest1` on 272 of the 340 inputs; the corrected version emitted no message. Wall-clock totals: original 27 s, corrected 39 s; the slowest single input took 0.4 s (original) and 1.7 s (corrected).

| Suite | Failing tests (all predicted by the review) | Passing |
|---|---|---|
| R1 (14) | Print protection; comparator; negative branch; grouped complex branch; wrapper complex product; `Factorc[Sqrt[t^2]]`; zero multiplier | 7: rationalizer (2), direct and multiplier success, `Sqrt[t + t^2]` (*not* predicted to pass), empty seed list, all-levels |
| R2 (15) | comparator; negative branch; complex 3/2 power; merged product; `Factorc[Sqrt[z^2]]`; caller's `f`; numerator `x` | 4 on the original (Print protection[1], worked example, rationalizer, all-level control); the 4 tests of its guarded component pass when the component is loaded (Section 5.5) |
| R3 (20) | negative core target; complex 3/2; merged product (core and wrapper); `Factorc[Sqrt[-x-y]]`; zero multiplier; comparator; stale slice; cap; Print protection | 10: classical success, rationalizer (2), non-nested input, wrapper modes with `Identity`, and all 5 tests of its proposed kernel |

Table 15: The reviews' regression suites executed against the original program.

| Family (inputs) | Example | Cause |
|---|---|---|
| A. Principal root of a negative radicand, index $\geq 5$ (8) | $(41-29\sqrt{2})^{1/5} = (-1)^{1/5}(\sqrt{2}-1)$ | the core solves only the factor of $x^r - \rho$ shared with the principal root's minimal polynomial; for $r \geq 5$ `Solve` returns `Root` objects or nested surds, certified but not cheaper. Index 3 and 4 work. |
| B. Composite index, answer of larger degree than the radicand (4) | $\sqrt[6]{7\sqrt[3]{20}-19} = \sqrt[3]{5/3} - \sqrt[3]{2/3}$ | the certified candidate is $\gamma^{1/3}$ with $\gamma = \beta^3 \in \mathbb{Q}(\rho)$, same depth as the input, hence discarded; no index reduction, no recursion on rejected candidates. |
| C. Factorizer extracts a factor with irrational root (1) | $\sqrt[6]{1296+880\sqrt{2}+720\sqrt{3}+528\sqrt{6}}$ | `Factorc` pulls out 16, leaving $2^{2/3}(\cdots)^{1/6}$ of degree 12; with `"Factor"->False` the input is denested in 0.1 s. |
| D. Cancellation across summands (1) | $\sqrt{1+\sqrt{3}}+\sqrt{3+3\sqrt{3}}-\sqrt{10+6\sqrt{3}} = 0$ | summands are independent problems and are individually non-denestable (Section 8, item 3). |
| E. Time budget not enforced inside a trial (1) | $\sqrt{(\sqrt{2}+\sqrt{3}+\sqrt{5}+\sqrt{7}+\sqrt{11})^2}$ expanded | one `PolynomialGCD`/`Solve` over eleven surds exceeds 150 s; the budget is checked only between trials, so guarantee (iv) of Section 7.4 fails here. Four primes take 2.9 s. |

Table 21: Denestable inputs that the corrected version leaves unchanged (5 September 2026).

# A The original source, with line numbers

The file `src/original/Strad.wl` is reproduced verbatim (585 physical lines). All "lines $n$–$m$" references in the text refer to these numbers.

```
ClearAll[Strad];
Unprotect[Print];
Strad[eaf__] := Block[{Print = (Null &)}, DenestRadicals3[eaf]];
ClearAll[DenestRadicals3];
DenestRadicals3[expr_, alllevels_ : False, func_ : denest11] :=
 Block[{f, mod, level, problems, i1, i2},
  If[alllevels =!= True,
   level = ReplaceAll,
   level = ReplaceRepeated
   ];
  mod = level[Factorc[expr],
    radicand_?AlgebraicQ^pow_Rational :> Block[{factored, SumQ, i, j},
      SumQ[expression_] :=
       MemberQ[
        expression, _Plus | Complex[Except[0], _], {0, Infinity}];
      factored = FactorTermsList[radicand];
      factored[[2]] =
       Flatten[{Replace[Factorc[factored[[2]]],
          prod_Times :> List @@ prod]}];
      factored[[1]] =
       DeleteCases[{#, ComplexExpand[factored[[1]]/#]} &[
         GCD[Re[factored[[1]]], Im[factored[[1]]]]], 1];
      factored[[1]] = #[[1]]^#[[2]] & /@
        Flatten[FactorInteger /@ factored[[1]], 1];
      factored = Flatten[factored];
      factored =
       factored //.
        list : {a___, expr1_, b___, expr2_, c___} :> {a, b, c,
           ComplexExpand[expr1*expr2]} /; And[SumQ[expr1], SumQ[expr2]];
      j = 1;
      For[{i = 1}, i <= Length[factored], i++,
       If[N[Re[factored[[i]]], 100] < 0,
        factored[[i]] = -factored[[i]];
        j *= -1
        ]
       ];
      factored = DeleteCases[Prepend[factored, j], 1];
      f[Simplify[
        Times @@ (Simplify[RationalizeDenominator[Together[#]]] & /@
           factored)], pow]
      ]
    ];
  mod = mod //.
    prod : Times[a___, expr1 : f[rad1_, pow1_], b___,
       expr2 : f[rad2_, pow2_], c___] :>
     Times[a, b, c, f[(rad1^pow1)*(rad2^pow2)]] /;
      And[! FreeQ[rad1, Power], ! FreeQ[rad2, Power]];
  mod = level[mod, f[rad_, pow_] :> f[rad^pow]];
  mod = mod //.
    prod : Times[a___, expr1 : f[rad1_], b___, expr2 : f[rad2_],
       c___] :>
     Times[a, b, c, f[rad1*rad2]] /;
      And[And[Re[#] != 0, Im[#] != 0] &[N[rad1, 10]],
       And[Re[#] != 0, Im[#] != 0] &[N[rad2, 10]]];
  problems = DeleteDuplicates[Cases[mod, f[_], {0, Infinity}]];
  Print[problems];
  Print[];
  If[alllevels =!= True,
   mod =
    Simplify[
     mod /. ((# -> Replace[#, f[val_] :> func[val]]) & /@ problems)],
   For[{i1 = 1}, MemberQ[problems, f[__]] === True, i1++,
```

```
    i1 = Mod[i1, Length[problems], 1];
    If[MatchQ[problems[[i1]], f[problem__] /; FreeQ[problem, f[__]]],
     problems[[i1]] = problems[[i1]] -> (problems[[i1]] /. f -> func);
     For[{i2 = 1}, i2 <= Length[problems], i2++,
      If[Head[problems[[i2]]] =!= Rule,
       problems[[i2]] = problems[[i2]] //. problems[[i1]];
       ]
      ];
     ];
    ];
   mod = Simplify[mod //. problems];
   ];
  Return[mod];
  ]

ClearAll[denest11];
denest11[problem_, forcedmultipliers_ : 0] :=
 Block[{radicand, outerpower, reductionpower, extrapower,
   multiplierstack, multipliers, multiplier, terms, history, attempt,
   solution, updatehistory, stackchange, addmultipliers,
   trymultiplier, newmultipliers, x, i1, i2, i3},
  If[! MemberQ[problem,
     Power[inner_?(Function[var, MemberQ[var, Power[a_, b_], Infinity]]),
       outer_], {0, Infinity}],
   Return[problem];
   ];
  Print["problem  ", problem];
  reductionpower = (LCM @@ (maxnestdepth[problem,
        MatchQ[#, Power[_, _Rational]] &] /. {Power[base_, exp_], _} :>
         Denominator[exp]));
  outerpower = 1/reductionpower;
  radicand = problem^reductionpower;
  extrapower =
   Times @@ ((LCM @@ #) & /@ (DeleteCases[
         Cases[nestdepthsort[radicand,
           MatchQ[#, Power[_, _Rational]] &],
          patt : {{expr_, nest_}, Repeated[{_, _}, {0, Infinity}]} /;
           nest >= 2], {Except[Power[_, _Rational]], _}, {2}] /.
       patt : {Power[base_, exp_], nest_} :> Denominator[exp]));
  Print["outer root: ", reductionpower, "  extra root: ", extrapower];
  Print[];

  (*tries a multiplier
  returns {<boolean progress>, {multiplier, minpoly, degree}, [answer]}*)
  trymultiplier[radicand_, outerpower_, multiplier_, best_,
     worseknown_] /; MatchQ[outerpower, Rational[1, _Integer]] :=
   Block[{minpoly, degree, polygcd, possibilities, result, expected,
     arg1},
    minpoly = MinimalPolynomial[(multiplier*radicand)^outerpower];
    degree = Exponent[minpoly[arg1], arg1];
    result = {False, {multiplier, minpoly, degree}};
    Print["multiplier: ", multiplier, "   minpoly degree: ", degree];
    If[Or[degree < best, GCD[degree, best] < best],
     polygcd =
      PolynomialGCD[minpoly[arg1],
       arg1^(1/outerpower) - multiplier*radicand,
       Extension -> Automatic];
     If[And[Exponent[polygcd, arg1] < (1/outerpower), polygcd =!= 1],
      possibilities =
       Flatten[Outer[Times,
         arg1/(multiplier^outerpower) /. Solve[polygcd == 0, arg1],
         Exp[2*I*\[Pi]*Range[0, (1/outerpower) - 1, 1]/(1/outerpower)]]];
      result =
       DeleteDuplicates[
        Select[possibilities, PossibleZeroQ[# - radicand^outerpower] &]];
      If[Length[result] > 1,
       Print[Length[result], " possible solutions, taking the first"];
       Print[result];
```

```
        result = Take[result, 1];
        ];
       (*progress because a denesting occurred*)
       result = {True, {multiplier, minpoly, degree}, result[[1]]},
       If[polygcd =!= 1,
        If[
         Or[degree < best,
          And[GCD[degree, best] < best, worseknown =!= True]],
         (*progress because degree of minpoly has decreased*)
         result = {True, {multiplier, minpoly, degree}}
         ],
        Print[
         "PolynomialGCD error: ", {radicand, outerpower, multiplier,
          minpoly}]
        ]
       ],
      If[And[worseknown =!= True, degree != best],
       (*progress because this multiplier has greater degree than a pre\
vious multiplier*)
       Print["previous progress identified"];
       result = {True, {multiplier, minpoly, degree}};
       ]
      ];
     Return[result];
     ];

   (*form for attempts is {multiplier, minpoly, degree},
   attempt2 is Null if only one attempt is available*)
   newmultipliers[smaller_, larger_, reductionpower_] :=
    Block[{done, new, primes, multiplierprimes, minprimes, maxprimes,
      primeratio, multipliercap, arg1},
     minprimes = 5;
     maxprimes = 14;
     primeratio = 100;
     multipliercap = 1000;

     (*two possibilites: reduction in degree,
     reduction only in GCD of degrees*)
     If[Or[larger === Null,
       GCD[smaller[[-1]], larger[[-1]]] ==
        Min[smaller[[-1]], larger[[-1]]]],
      (*one degree is a multiple of the other, usual case*)
      primes =
       DeleteCases[
        FactorInteger[Discriminant[smaller[[2]][arg1], arg1],
         Automatic], {-1, 1}];
      primes =
       Sort[DeleteCases[primes,
         factor : {prime_, _} /;
          And[! PrimeQ[prime], (Print["composite factor: ", factor];
            True)]], #1[[1]] <= #[[2]] &];
      primes = Flatten[primes[[All, 1]]];
      multiplierprimes = primes;
      For[{i3 = Length[primes]},
       And[i3 > minprimes,
        multiplierprimes[[i3]]/multiplierprimes[[i3 - 1]] > primeratio],
        i3--,
       multiplierprimes[[i3]] = 0;
       ];
      multiplierprimes =
       DeleteCases[multiplierprimes, 0][[
        1 ;; Min[
          Max[minprimes, Ceiling[Log[multipliercap]/Log[reductionpower]]],
           Length[multiplierprimes]]]];
      new =
       smaller[[1]]*
        Drop[Union[
          Divisors[(Times @@ multiplierprimes)^(reductionpower - 1)],
```

```
199           primes], 1];
200       ,
201       (*different degree reductions in each degree from a larger degree\
202 *)
203       Print[
204        "GCD of smallest degrees is smaller than any observed degree: \
205 ", {smaller[[{1, -1}]], larger[[{1, -1}]]}];
206       new = smaller[[1]]*larger[[1]]*{1};
207       Print[PolynomialGCD @@ {smaller[[2]][x], larger[[2]][x]}];
208       ];
209      new = {{False, smaller}, ComplexitySort[new]};
210      Return[new];
211      ];
212
213    (*compute initial set of multipliers*)
214    If[ListQ[forcedmultipliers],
215     multipliers = {{False, {0, Infinity, Infinity}}, forcedmultipliers},
216     (*old, may be better*)
217     (*
218     terms=DeleteCases[Replace[#,{Times[a___,
219     Alternatives[_Rational,_Integer],b___]\[RuleDelayed]a*b,
220     Alternatives[_Rational,_Integer]\[Rule]0,Complex[a_,
221     b_]\[RuleDelayed]Sign[b]*I}]&/@(List@@Expand[
222     RationalizeDenominator[radicand]]),0];
223     *)
224     terms =
225      DeleteCases[
226       Replace[#, {Times[a___, Alternatives[_Rational, _Integer],
227             b___] :> a*b, Alternatives[_Rational, _Integer] -> 0,
228           Complex[a_, b_] :> Sign[b]*I}] & /@ (List @@
229          Expand[RationalizeDenominator[radicand]]), 0];
230     multipliers = Prepend[Replace[If[Head[#] === Times,
231
232           Times @@ (Function[{var},
233               var^(maxnestdepth[var,
234                    Function[{var2},
235                     MatchQ[var2, Power[_, _Rational]]]][[
236                   1]] /. {{Power[base_,
237                      exp_], _} :> (Denominator[exp] - Numerator[exp])/
238                     Numerator[exp], {Complex[
239                      0, _], _} :> -1})] /@ (List @@ #)),
240           #^(maxnestdepth[#,
241                Function[{var}, MatchQ[var, Power[_, _Rational]]]][[
242               1]] /. {{Power[base_,
243                  exp_], _} :> (Denominator[exp] - Numerator[exp])/
244                 Numerator[exp], {Complex[0, _], _} :> -1})
245           ],
246          Times[a___, Alternatives[_Rational, _Integer], b___] :>
247           a*b] & /@ Sort[terms, (Re[#1] <= Re[#2]) &], 1];
248     multipliers = {{True, {0, Infinity, Infinity}}, multipliers};
249     ];
250
251    solution = problem;
252    history = {{0, Infinity, Infinity}};
253    (*multiplierstack form {{{<do full set>,{<multiplier,minpoly,
254    degree for generating this set>}},{<multipliers>}},...}*)
255    multiplierstack = {multipliers};
256    (*add power set of multipliers as second element in multiplierstack*)
257    (*multiplierstack=Append[multiplierstack,
258    {multiplierstack[[1,1]],ComplexitySort[DeleteCases[Union[(Times@@#)&/@
259    Subsets[DeleteCases[multipliers[[2]],1]]],mult_/;MemberQ[
260    multipliers[[2]],mult]]]}];*)
261    Catch[
262     For[{i1 = 1}, i1 <= Length[multiplierstack], i1++,
263      (*compute new multipliers from data if not already done*)
264      If[ListQ[multiplierstack[[i1, 2]]] =!= True,
265       multiplierstack[[i1]] =
266         multiplierstack[[i1, -2]] @@ (multiplierstack[[i1, -1]]);
```

```
 ];
multipliers = multiplierstack[[i1]];
stackchange = False;
If[multipliers[[1, 2]] =!= history[[1]],
 Print["multiplier count: ", Length[multipliers[[2]]], "  from: ",
   multipliers[[1, 2, {1, -1}]]],
 If[! ListQ[forcedmultipliers],
  Print["multiplier count: ", Length[multipliers[[2]]],
    "  multipliers: ", multipliers[[2]]];,
  Print["multiplier count: ", Length[multipliers[[2]]]];
  ]
 ];
For[{i2 = 1}, i2 <= Length[multipliers[[2]]], i2++,
 multiplier = multipliers[[2, i2]];
 updatehistory = False;
 addmultipliers = False;
 (*try a multiplier, different based on history*)
 If[And[Length[history] >= 3, history[[-2]] =!= history[[1]]],
  attempt =
   trymultiplier[radicand, outerpower, multiplier,
    history[[-1, -1]], True],
  attempt =
    trymultiplier[radicand, outerpower, multiplier,
     history[[-1, -1]], False];
  ];
 multiplierstack[[i1, 2, i2]] = {multiplier, attempt[[2, -1]]};

 If[And[multipliers[[1, 1]] === True,
   attempt[[2, -1]] <= history[[-1, -1]]],
  addmultipliers = True
  ];
 (*check if progress has been made on this attempt,
 no action taken if no progress*)
 If[attempt[[1]] === True,
  (*history will be updated*)
  updatehistory = True;

  (*decide if a stackchange will be made*)
  If[history[[-1]] =!= history[[1]],
   stackchange = True;
   ];

  If[ListQ[attempt[[-1]]] =!= True,
   (*a solution was obtained*)
   Print["success: ",
    Join[attempt[[2, {1, -1}]],
     If[IntegerQ[multiplier], {FactorInteger[multiplier]}, {}]]];
   solution = Simplify[attempt[[-1]]];
   If[
    Or[history[[-1]] === history[[1]],
     history[[2, -1]]/attempt[[2, -1]] >= reductionpower],
    (*not worth trying for additional denesting, exit immediately*)
    history = Append[history, attempt[[2]]];
    Throw[, "solution"],
    (*further denesting will be attempted*)
    Print["attempting additional denesting, current solution is:"];
    Print[solution];
    ];
   ];

  If[
   And[history[[-1]] =!= history[[1]],
    Or[attempt[[2, -1]] < history[[-1, -1]],
     history[[-2]] === history[[1]]]],
   (*improvement has occured, new multipliers will be created*)
   Print["improvement  ", attempt[[2, {1, -1}]]];
   addmultipliers = True;
   ];
```

```
335       ];
336
337      (*add new multipliers if needed*)
338      If[addmultipliers === True,
339       Block[{smaller, larger, done, new},
340        If[attempt[[2, -1]] <= history[[-1, -1]],
341         smaller = attempt[[2]];
342         larger = history[[-1]],
343         smaller = history[[-1]];
344         larger = attempt[[2]];
345         ];
346        new = {{False, smaller},
347          newmultipliers, {smaller,
348           If[larger =!= history[[1]], larger, Null], reductionpower}};
349
350        Print["adding new multipliers from: ", multiplier];
351        (*update multiplierstack with new multipliers*)
352        If[multipliers[[1, 1]] === False,
353         (*use new multipliers immediately*)
354         done = {multipliers[[1]], multipliers[[2, 1 ;; i2]]};
355         multiplierstack = Insert[multiplierstack, new, i1 + 1];
356         If[i2 < Length[multipliers[[2]]],
357
358          multiplierstack =
359            Insert[multiplierstack, {multipliers[[1]],
360              multipliers[[2, i2 + 1 ;; -1]]}, i1 + 2];
361          ];
362         multiplierstack[[i1]] = done,
363         (*use new multipliers later*)
364         stackchange = False;
365         For[{i3 = Length[multiplierstack]}, i3 >= i1, i3--,
366
367          If[Or[multiplierstack[[i3, 1, 2]] === history[[1]],
368            new[[1, 2, -1]] > multiplierstack[[i3, 1, 2, -1]],
369            And[new[[1, 2, -1]] === multiplierstack[[i3, 1, 2, -1]],
370             LeafCount[new[[1, 2, 1]]] >=
371              LeafCount[multiplierstack[[i3, 1, 2, 1]]]]],
372           multiplierstack = Insert[multiplierstack, new, i3 + 1];
373           Break[];
374           ]
375          ];
376         ];
377        ]
378       ];
379
380      (*update history if needed*)
381      If[updatehistory === True,
382       For[{i3 = Length[history]}, i3 >= 1, i3--,
383         If[history[[i3, -1]] > attempt[[2, -1]],
384          history = Insert[history, attempt[[2]], i3 + 1];
385          Break[];
386          ]
387         ];
388       ];
389
390      (*break inner loop if stack has changed*)
391      If[stackchange === True,
392       Break[];
393       ];
394      ];(*end of inner loop*)
395
396     (*add additional multipliers after first pass (if no forced multip\
397  liers)*)
398     If[And[i1 === 1, ! ListQ[forcedmultipliers]],
399      Block[{best, worst, extra, f},
400       best = Min[multiplierstack[[i1, 2, All, 2]]];
401       worst = Max[multiplierstack[[i1, 2, All, 2]]];
402       extra =
```

```
        DeleteCases[
         DeleteDuplicates[
          Replace[(Distribute[
                f @@ (#^Range[0, 1] & /@ (Cases[
                     multiplierstack[[i1, 2]], {mult_, degree_} /;
                     degree < worst][[All, 1]])),
                List] //. {f[a___, Power[base_Times, exp_], b___] :>
                 f[a, b, Sequence @@ ((List @@ base)^exp)],
                f[a___, prod_Times, b___] :> f[a, b, Sequence @@ prod],
                 f[a___, Power[base_, exp_], b___] :>
                 f[a, b, (-1)^exp, (-base)^exp] /;
                  And[base != -1, Re[base] < 0],
                f[a___, Power[base_, Rational[num1_, denom_]], b___,
                  Power[base_, Rational[num2_, denom_]], c___] :>
                 f[a, b, c, base^(Mod[num1 + num2, denom]/denom)]}) /.
              f -> Times, Times[a___, _Integer, b___] :> a*b, {1}] //.
            Power[base_, Rational[a_, b_]] :> base^(Mod[a, b]/b)],
          Alternatives @@ multiplierstack[[i1, 2, All, 1]]];
        If[Length[extra] >= 1,
         Print["adding multipliers based on progress in initial guess"];
         multiplierstack =
          Insert[multiplierstack, {multiplierstack[[i1, 1]], extra},
           i1 + 1];
         ];
        ]
       ];

      ](*end of outer loop*),
     "solution"
     ];(*end of Catch, used to break the loop when a solution is found*)

    Print["history: ", #[[{1, -1}]] & /@ history];
    If[solution =!= problem,
     Print[solution],
     Print["no progress"];
     ];
    Print[];
    Return[solution];
    ]

ClearAll[RationalizeDenominator];
RationalizeDenominator[expr_] :=
 Block[{conjugate, denom, x},
  denom = Denominator[expr];
  conjugate = MinimalPolynomial[denom];
  Expand[
    Numerator[expr]*PolynomialQuotient[conjugate[x], x - denom, x] /.
     x -> 0]/(-1*conjugate[0])
  ]

ClearAll[AlgebraicQ];
AlgebraicQ[num_] := Element[num, Algebraics]

ClearAll[nestdepths]
nestdepths[expr_, func_] :=
 Block[{inner},
  If[Depth[expr] === 1,
   {expr, If[func[expr], 1, 0]},
   inner = Map[nestdepths[#, func] &, (List @@ expr), {1}];
   {expr,
    Sequence @@
     inner, ((Max @@ ((#[[-1]]) & /@ inner)) + If[func[expr], 1, 0])}
   ]
  ]

ClearAll[nestdepthsort]
nestdepthsort[expr_, func_] :=
 Block[{data},
```

```
  data = nestdepths[expr, func];
  data =
   data //.
    patt : {a : Except[_List], b : __, c : Except[_List]} :> {{a, c},
      b};
  data = data //. patt : {rep : Repeated[{_, _}]} :> Sequence @@ patt;
  data = Gather[{data}, (#1[[2]] == #2[[2]]) &];
  data = Sort[data, (#1[[1, 2]] >= #2[[1, 2]]) &];
  data
  ]

ClearAll[maxnestdepth]
maxnestdepth[expr_, func_] :=
 Block[{data, max},
  data = nestdepths[{expr}, func][[2]];
  max = data[[-1]];
  data =
   FixedPoint[
    DeleteCases[# /. {a_, b_} :> {} /; b < max, {}, Infinity] &, data];
  data =
   If[And[ListQ[#], Length[#] > 2], Sequence @@ (#[[2 ;; -2]]), #] & /@
     data;
  If[Length[data] > 2,
   data = data[[2 ;; -2]],
   data = {data}
   ];
  data
  ]

ClearAll[ComplexitySort];
ComplexitySort[expr_] :=
  Return[Sort[expr,
    Switch[LeafCount[#1] - LeafCount[#2], a_ /; a < 0, True,
      a_ /; a > 0, False, 0, Abs[#1] <= Abs[#2]] &]];

ClearAll[Factorc]
Factorc[expr_] :=
 Flatten[
   fullfactor[
     expr] //. {pow : {a : Except[_List], b : Except[_List]} :> {a^b},
      prod : {rep : (Repeated[{Except[_List]}, {2,
            Infinity}])} :> {Times[rep]},
     sum : {rep : Repeated[{{Except[_List]}}, {2, Infinity}]} :>
      Plus[rep],
     pow2 : {{a : Except[_List]}, b : Except[_List]} :> {a^b},
     pow3 : {{{a : Except[_List]}}, b : Except[_List]} :> {a^b}}][[1]]

ClearAll[fullfactor]
fullfactor[expr_] :=
 Block[{factorlist, i = 0, j = 0, done = False},
  factorlist = FactorList[expr];
  While[! done && j < 5,
   j++;
   done = True;
   For[{i = 1}, i <= Length[factorlist], i++,
    Which[
     Head[factorlist[[i, 1]]] === Power,
     done = False;
     factorlist[[
       i]] = {{factorlist[[i, 1, 1]],
        factorlist[[i, 1, 2]]*factorlist[[i, 2]]}},
     And[
      MemberQ[{Integer, Rational}, Head[factorlist[[i, 1]]]], !
       PrimeQ[factorlist[[i, 1]]], Abs[factorlist[[i, 1]]] =!= 1],
     done = False;
     factorlist[[i]] = {#[[1]], #[[2]]*factorlist[[i, 2]]} & /@
       FactorInteger[factorlist[[i, 1]]],
     Head[factorlist[[i, 1]]] === Plus,
```

```
      done = False;
      factorlist[[i]] =
       combine[fullfactor /@
         Replace[factorlist[[i, 1]], Plus[a_, b__] :> {a, b}],
        factorlist[[i, 2]]],
      True,
      factorlist[[i]] = {factorlist[[i]]}
      ]
     ];
    factorlist = Flatten[factorlist, 1]
    ];
   factorlist
   ]

ClearAll[combine];
combine[terms_List, pow_] :=
 Block[{simpler = terms, base, bases, positions, factor, i, j, k},
  For[{i = 1}, i <= Length[terms], i++,
   base = terms[[i]];
   For[{j = 1}, j < Length[base], j++,
    For[{k = j + 1}, k <= Length[base], k++,
     If[base[[j, 1]] === base[[k, 1]],
      base[[j]] = {base[[j, 1]], base[[j, 2]] + base[[k, 2]]};
      base[[k]] = {0, 0}
      ]
     ];
    If[base[[j, 1]] === 1,
     base[[j]] = {0, 0}
     ]
    ];
   simpler[[i]] = Delete[base, Position[base, {0, 0}]];
   ];
  bases = Intersection @@ Map[#[[1]] &, simpler, {2}];
  For[{i = 1}, i <= Length[bases], i++,
   positions =
    Position[#, {bases[[i]], Except[_List]}, 1] & /@ simpler;
   factor =
    Min[Extract[simpler[[#]], positions[[#]]][[1, 2]] & /@
      Range[1, Length[simpler], 1]];
   For[{j = 1}, j <= Length[simpler], j++,
    simpler[[j, Sequence @@ (positions[[j, 1]]), 2]] -= factor
    ];
   bases[[i]] = {bases[[i]], factor*pow};
   ];
  Append[bases, {simpler, pow}]
  ]
```

# B The corrected source

The file `src/corrected/StradFixed.wl`. Load it with `Get["StradFixed.wl"]`; the public symbols live in the context `RadicalDenest‘`.

```
(* ::Package:: *)

(* StradFixed.wl -- a corrected and hardened version of src/original/Strad.wl

   The architecture of the original (wrapper -> marker -> multiplier search ->
   minimal polynomial / polynomial GCD -> roots-of-unity orbit) is retained.
   The changes are confined to correctness, robustness and resource control:

     * every accepted candidate is certified, with exact algebra, to equal the
       ORIGINAL problem (not the principal root of its power);
     * the custom factorizer only applies the power identities
       (a^b)^c -> a^(bc) and (F S)^p -> F^p S^p when they are provably valid,
       and its output is certified anyway;
     * factoring and marking are restricted to exact numeric algebraic
       subexpressions, so symbolic hosts are left untouched;
     * the prime comparator, the stale-length slice, the Ceiling[Log] cap and
       the {}[[1]] access are fixed; the multiplier cap is a real cap;
     * the search has a trial budget, a time budget and a visited set;
     * private context symbols replace the marker ‘f‘ and the variable ‘x‘;
     * no Unprotect[Print]; diagnostics go through a "Verbose" option;
     * a result is only returned if it is certified equal to the input AND
       strictly cheaper under an explicit radical-complexity cost.

   Author: analysis session of 5 September 2026. *)

BeginPackage["RadicalDenest‘"];

Strad::usage = "Strad[expr, opts] denests exact algebraic radicals occurring in expr and returns an
    expression certified equal to expr. Strad[expr, True] processes all nesting levels.";
DenestRadicals::usage = "DenestRadicals[expr, opts] is the option-driven wrapper behind Strad.
    DenestRadicals[expr, allLevels] is the legacy positional form.";
DenestCore::usage = "DenestCore[problem, opts] runs the multiplier search on a single exact nested
    radical and returns a certified equal expression (or problem itself).";
RationalizeDenominator::usage = "RationalizeDenominator[expr] rewrites expr so that its denominator is
    rational, using the minimal polynomial of the denominator.";
RadicalDepth::usage = "RadicalDepth[expr] is the maximal number of nested rational-power nodes on a path
     of expr.";
RadicalCost::usage = "RadicalCost[expr] is the lexicographic cost {#Root objects, radical depth, #
    radical nodes, leaf count} used to decide whether a rewrite is an improvement.";
ExactAlgebraicQ::usage = "ExactAlgebraicQ[e] is True when e is an exact numeric algebraic number.";
CertifiedEqualQ::usage = "CertifiedEqualQ[a, b] is True only when a - b is proved to be zero by exact
    algebra.";
Factorc::usage = "Factorc[expr] is the branch-safe version of the original custom factorizer.";

Options[Strad] = Options[DenestRadicals] = {
   "AllLevels" -> False, "Verbose" -> False, "Multipliers" -> Automatic,
   "MaxTrials" -> 400, "TimeBudget" -> 120, "MultiplierCap" -> 1000,
   "CertifyTime" -> 20, "Solver" -> Automatic, "Factor" -> True};
Options[DenestCore] = {
   "Verbose" -> False, "Multipliers" -> Automatic, "MaxTrials" -> 400,
   "TimeBudget" -> 120, "MultiplierCap" -> 1000, "CertifyTime" -> 20};

Begin["‘Private‘"];

(* ------------------------------------------------------------------ *)
(* logging, predicates, certification                                 *)
(* ------------------------------------------------------------------ *)

$verbose = False;
$certifyTime = 20;
log[args___] := If[TrueQ[$verbose], Print[args]];

```

```
ExactAlgebraicQ[e_] := TrueQ[NumericQ[e] && Precision[e] === Infinity &&
    Quiet[Check[Element[e, Algebraics], False]]];

exactRealQ[e_] := ExactAlgebraicQ[e] && TrueQ[Quiet[Element[e, Reals]]];
positiveRealQ[e_] := exactRealQ[e] && TrueQ[Quiet[e > 0]];

(* numeric prefilter (cheap rejection) followed by exact certification *)
CertifiedEqualQ[a_, b_] := Module[{d, num, r},
   If[! ExactAlgebraicQ[a] || ! ExactAlgebraicQ[b], Return[False]];
   d = a - b;
   num = Quiet[N[d, 30]];
   If[NumberQ[num] && Abs[num] > 10^-20, Return[False]];
   r = Quiet[TimeConstrained[Check[RootReduce[d] === 0, $Failed], $certifyTime, $Failed]];
   If[r === True, Return[True]];
   If[r === False, Return[False]];
   r = Quiet[TimeConstrained[Check[PossibleZeroQ[d, Method -> "ExactAlgebraics"], $Failed], $certifyTime
     , $Failed]];
   TrueQ[r]];

(* ---------------------------------------------------------------- *)
(* representation cost                                              *)
(* ---------------------------------------------------------------- *)

RadicalDepth[e_] := If[AtomQ[e], 0,
   If[MatchQ[e, Power[_, _Rational]], 1 + RadicalDepth[First[e]],
    Max[Prepend[RadicalDepth /@ (List @@ e), 0]]]];

RadicalCost[e_] := {Count[e, _Root | _AlgebraicNumber, {0, Infinity}],
   RadicalDepth[e], Count[e, Power[_, _Rational], {0, Infinity}], Count[e, _?AtomQ, {-1}], LeafCount[e
    ]};

cheaperQ[new_, old_] := Order[RadicalCost[new], RadicalCost[old]] === 1;
pickCheapest[list_List] := First[SortBy[list, RadicalCost]];

safeSimplify[e_] := Quiet[TimeConstrained[Simplify[e], $certifyTime, e]];

(* ---------------------------------------------------------------- *)
(* denominator rationalization (valid for nonzero algebraic denominators) *)
(* ---------------------------------------------------------------- *)

RationalizeDenominator[expr_] := Module[{x, e, num, denom, minpoly, q},
   e = Together[expr];
   denom = Denominator[e]; num = Numerator[e];
   If[denom === 1 || ! ExactAlgebraicQ[denom], Return[expr]];
   minpoly = Quiet[Check[MinimalPolynomial[denom, x], $Failed]];
   If[! PolynomialQ[minpoly, x], Return[expr]];
   q = PolynomialQuotient[minpoly, x - denom, x] /. x -> 0;
   Expand[num*q]/(-(minpoly /. x -> 0))];

(* ---------------------------------------------------------------- *)
(* structural helpers                                               *)
(* ---------------------------------------------------------------- *)

radicalQ[e_] := MatchQ[e, Power[_, _Rational]];
nestedRadicalQ[e_] := RadicalDepth[e] >= 2;

(* LCM of exponent denominators of the rational-power nodes of maximal depth *)
reductionExponent[e_] := Module[{d = RadicalDepth[e], nodes},
   nodes = Select[Cases[e, Power[_, _Rational], {0, Infinity}], RadicalDepth[#] == d &];
   If[nodes === {}, 1, LCM @@ (Denominator[Last[#]] & /@ nodes)]];

summands[e_] := If[Head[e] === Plus, List @@ e, {e}];
factorsOf[e_] := If[Head[e] === Times, List @@ e, {e}];

complexitySort[list_] := SortBy[list, {LeafCount[#], Quiet[N[Abs[#], 20]]} &];

(* ---------------------------------------------------------------- *)
(* branch-safe custom factorizer                                    *)
```

```
(* ---------------------------------------------------------------------- *)

(* (a^b)^c -> a^(b c) is valid when c is an integer or b is real with -1 < b <= 1 *)
powerMergeSafeQ[b_, c_] := IntegerQ[c] || (exactRealQ[b] && TrueQ[-1 < b <= 1]);

Factorc[expr_] := Module[{r},
   r = Quiet[Check[factorcRaw[expr], expr]];
   If[ExactAlgebraicQ[expr], If[CertifiedEqualQ[r, expr], r, expr], r]];

factorcRaw[expr_] :=
  Flatten[fullfactor[expr] //. {
      pow : {a : Except[_List], b : Except[_List]} :> {a^b},
      prod : {rep : (Repeated[{Except[_List]}, {2, Infinity}])} :> {Times[rep]},
      sum : {rep : Repeated[{{Except[_List]}}, {2, Infinity}]} :> Plus[rep],
      pow2 : {{a : Except[_List]}, b : Except[_List]} :> {a^b},
      pow3 : {{{a : Except[_List]}}, b : Except[_List]} :> {a^b}}][[1]];

fullfactor[expr_] := Module[{factorlist, i = 0, j = 0, done = False},
   factorlist = FactorList[expr];
   While[! done && j < 5,
    j++;
    done = True;
    For[i = 1, i <= Length[factorlist], i++,
     Which[
      Head[factorlist[[i, 1]]] === Power && powerMergeSafeQ[factorlist[[i, 1, 2]], factorlist[[i, 2]]],
      done = False;
      factorlist[[i]] = {factorlist[[i, 1, 1]], factorlist[[i, 1, 2]]*factorlist[[i, 2]]},
      And[MemberQ[{Integer, Rational}, Head[factorlist[[i, 1]]]], ! PrimeQ[factorlist[[i, 1]]], Abs[
     factorlist[[i, 1]]] =!= 1],
      done = False;
      factorlist[[i]] = {#[[1]], #[[2]]*factorlist[[i, 2]]} & /@ FactorInteger[factorlist[[i, 1]]],
      Head[factorlist[[i, 1]]] === Plus,
      done = False;
      factorlist[[i]] = combine[fullfactor /@ (List @@ factorlist[[i, 1]]), factorlist[[i, 2]]],
      True,
      factorlist[[i]] = {factorlist[[i]]}]];
    factorlist = Flatten[factorlist, 1]];
   factorlist];

(* common-factor extraction from a sum raised to the power pow:
   (F S)^pow -> F^pow S^pow only when pow is an integer, or F > 0, or S > 0 *)
combine[terms_List, pow_] := Module[{simpler = terms, base, bases, positions, factor, i, j, k, extracted
    , remaining, fval, sval},
   For[i = 1, i <= Length[terms], i++,
    base = terms[[i]];
    For[j = 1, j <= Length[base], j++,
     For[k = j + 1, k <= Length[base], k++,
      If[base[[j, 1]] === base[[k, 1]] && base[[j]] =!= {0, 0},
       base[[j]] = {base[[j, 1]], base[[j, 2]] + base[[k, 2]]};
       base[[k]] = {0, 0}]];
     If[base[[j, 1]] === 1, base[[j]] = {0, 0}]];
    simpler[[i]] = DeleteCases[base, {0, 0}]];
   bases = Intersection @@ Map[#[[1]] &, simpler, {2}];
   If[bases === {}, Return[{{simpler, pow}}]];
   extracted = simpler;
   bases = Table[
     positions = Position[#, {bases[[i]], Except[_List]}, 1] & /@ extracted;
     factor = Min[Extract[extracted[[#]], positions[[#]]][[1, 2]] & /@ Range[Length[extracted]]];
     For[j = 1, j <= Length[extracted], j++,
      extracted[[j, Sequence @@ (positions[[j, 1]]), 2]] -= factor];
     {bases[[i]], factor},
     {i, Length[bases]}];
   fval = Times @@ (#[[1]]^#[[2]] & /@ bases);
   sval = Plus @@ ((Times @@ (#[[1]]^#[[2]] & /@ #)) & /@ extracted);
   If[IntegerQ[pow] || positiveRealQ[fval] || positiveRealQ[sval],
    Append[({#[[1]], #[[2]]*pow} & /@ bases), {extracted, pow}],
    {{simpler, pow}}]];

```

```
(* apply the factorizer only to maximal exact numeric non-atomic subexpressions *)
numericTargetQ[s_] := ! AtomQ[s] && ExactAlgebraicQ[s];
safeFactorAll[expr_] := If[numericTargetQ[expr], Factorc[expr], expr /. s_?numericTargetQ :> Factorc[s]];

(* ------------------------------------------------------------------ *)
(* the wrapper                                                        *)
(* ------------------------------------------------------------------ *)

Strad[expr_, opts : OptionsPattern[]] := DenestRadicals[expr, opts];
Strad[expr_, all : (True | False), opts : OptionsPattern[]] := DenestRadicals[expr, "AllLevels" -> all, opts];
Strad[l_List, rest___] := Strad[#, rest] & /@ l;

DenestRadicals[expr_, all : (True | False), opts : OptionsPattern[]] := DenestRadicals[expr, "AllLevels" -> all, opts];
DenestRadicals[l_List, rest___] := DenestRadicals[#, rest] & /@ l;

sumQ[expression_] := MemberQ[expression, _Plus | Complex[Except[0], _], {0, Infinity}];

DenestRadicals[expr_, opts : OptionsPattern[]] :=
  Block[{$verbose = OptionValue["Verbose"], $certifyTime = OptionValue["CertifyTime"]},
   Module[{level, mod, problems, solver, coreOpts, rules, result, candidates, i1, i2, iter, best},
    If[Precision[expr] =!= Infinity && ! FreeQ[expr, _Real | _Complex?InexactNumberQ], Return[expr]];
    level = If[TrueQ[OptionValue["AllLevels"]], ReplaceRepeated, ReplaceAll];
    solver = OptionValue["Solver"];
    coreOpts = FilterRules[{opts}, Options[DenestCore]];
    If[solver === Automatic, solver = recursiveCore[#, coreOpts] &];
    (* Root/AlgebraicNumber objects are first converted to radicals when possible *)
    mod = expr;
    If[! FreeQ[mod, _Root | _AlgebraicNumber],
     mod = Quiet[TimeConstrained[ToRadicals[mod], $certifyTime, mod]];
     If[! FreeQ[mod, _Root | _AlgebraicNumber], mod = expr]];
    mod = If[TrueQ[OptionValue["Factor"]], safeFactorAll[mod], mod];
    (* mark exact algebraic radicands raised to rational powers *)
    mod = level[mod, radicand_?ExactAlgebraicQ^pow_Rational :> normalizeRadicand[radicand, pow]];
    (* group products of two markers whose radicands both contain radicals *)
    mod = mod //. Times[a___, mark[rad1_, pow1_], b___, mark[rad2_, pow2_], c___] :>
       Times[a, b, c, mark[(rad1^pow1)*(rad2^pow2)]] /; And[! FreeQ[rad1, Power], ! FreeQ[rad2, Power]];
    mod = level[mod, mark[rad_, pow_] :> mark[rad^pow]];
    mod = mod //. Times[a___, mark[rad1_], b___, mark[rad2_], c___] :> Times[a, b, c, mark[rad1*rad2]]
     /;
       And[And[Re[#] != 0, Im[#] != 0] &[N[rad1, 10]], And[Re[#] != 0, Im[#] != 0] &[N[rad2, 10]]];
    problems = DeleteDuplicates[Cases[mod, mark[_], {0, Infinity}]];
    log["problems: ", problems];
    If[! TrueQ[OptionValue["AllLevels"]],
     rules = (# -> Replace[#, mark[val_] :> solver[val]]) & /@ problems;
     result = mod /. rules,
     (* all levels: resolve innermost markers first, with an iteration cap *)
     iter = 0;
     For[i1 = 1, MemberQ[problems, mark[__]] && iter < 20 Length[problems] + 20, i1++,
      iter++;
      i1 = Mod[i1, Length[problems], 1];
      If[MatchQ[problems[[i1]], mark[problem__] /; FreeQ[problem, mark[__]]],
       problems[[i1]] = problems[[i1]] -> (problems[[i1]] /. mark -> solver);
       For[i2 = 1, i2 <= Length[problems], i2++,
        If[Head[problems[[i2]]] =!= Rule, problems[[i2]] = problems[[i2]] //. problems[[i1]]]]]];
     result = mod //. Cases[problems, _Rule];
     result = result /. mark[v_] :> v];
    (* choose the cheapest certified representative among input, result, Simplify[result] *)
    candidates = DeleteDuplicates[{result, safeSimplify[result], denestInnerSurds[result]}];
    If[ExactAlgebraicQ[expr],
     candidates = Select[candidates, CertifiedEqualQ[#, expr] &]];
    candidates = Select[candidates, cheaperQ[#, expr] &];
    If[candidates === {}, expr, pickCheapest[candidates]]]];

(* the core, followed by one further all-level pass on a partial solution *)
$recursion = 0;
```

```
recursiveCore[problem_, coreOpts_] := Module[{sol, again},
   sol = DenestCore[problem, Sequence @@ coreOpts];
   If[sol =!= problem && RadicalDepth[sol] >= 2 && $recursion < 2,
    again = Block[{$recursion = $recursion + 1},
      DenestRadicals[sol, "AllLevels" -> True, Sequence @@ coreOpts]];
    If[cheaperQ[again, sol] && CertifiedEqualQ[again, problem], sol = again]];
   sol];

(* value-preserving normalization of a radicand (the original marker rule), certified *)
normalizeRadicand[radicand_, pow_] := Module[{factored, i, j, value},
   value = Quiet[Check[
      factored = FactorTermsList[radicand];
      factored[[2]] = Flatten[{Replace[Factorc[factored[[2]]], prod_Times :> List @@ prod]}];
      factored[[1]] = DeleteCases[{#, ComplexExpand[factored[[1]]/#]} &[GCD[Re[factored[[1]]], Im[
     factored[[1]]]]], 1];
      factored[[1]] = #[[1]]^#[[2]] & /@ Flatten[FactorInteger /@ factored[[1]], 1];
      factored = Flatten[factored];
      factored = factored //. {a___, expr1_, b___, expr2_, c___} :> {a, b, c, ComplexExpand[expr1*expr2
     ]} /; And[sumQ[expr1], sumQ[expr2]];
      j = 1;
      For[i = 1, i <= Length[factored], i++,
       If[TrueQ[N[Re[factored[[i]]], 100] < 0], factored[[i]] = -factored[[i]]; j *= -1]];
      factored = DeleteCases[Prepend[factored, j], 1];
      safeSimplify[Times @@ (safeSimplify[RationalizeDenominator[Together[#]]] & /@ factored)],
      $Failed]];
   If[value === $Failed || ! CertifiedEqualQ[value, radicand], value = radicand];
   mark[value, pow]];

(* ------------------------------------------------------------------ *)
(* quadratic surds: Sqrt[a + b Sqrt[c]] with rational a, b, c          *)
(* If a > 0 and a^2 - b^2 c = delta^2 with delta rational, then        *)
(*   Sqrt[a + b Sqrt[c]] = Sqrt[(a+delta)/2] + Sign[b] Sqrt[(a-delta)/2];   *)
(* the right-hand side is nonnegative, so it is the principal root.    *)
(* ------------------------------------------------------------------ *)

rationalQ[e_] := MatchQ[e, _Integer | _Rational];
(* named patterns are essential: two identical `_?rationalQ` factors would be
   combined by Times into (_?rationalQ)^(3/2) before matching *)
surdParts[rad_] := Replace[Expand[rad], {
    a_?rationalQ + b_?rationalQ Sqrt[c_?rationalQ] :> {a, b, c},
    a_?rationalQ + Sqrt[c_?rationalQ] :> {a, 1, c},
    _ :> $Failed}];
denestSurd[a_, b_, c_] := Module[{delta, s, u, v, r},
   If[a == 0 || b == 0 || c <= 0, Return[$Failed]];
   If[a < 0,
    If[TrueQ[a + b Sqrt[c] < 0], r = denestSurd[-a, -b, c]; Return[If[r === $Failed, $Failed, I r]],
     Return[$Failed]]];
   delta = a^2 - b^2 c;
   If[delta < 0, Return[$Failed]];
   s = Sqrt[delta];
   If[! rationalQ[s], Return[$Failed]];
   u = (a + s)/2; v = (a - s)/2;
   Sqrt[u] + Sign[b] Sqrt[v]];
denestSurdPower[rad_] := Module[{p = surdParts[rad], r},
   If[p === $Failed, Return[Sqrt[rad]]];
   r = denestSurd @@ p;
   If[r === $Failed, Sqrt[rad], r]];
denestInnerSurds[e_] := FixedPoint[Replace[#, Power[rad_, Rational[1, 2]] :> denestSurdPower[rad], {0,
     Infinity}] &, e, 4];

(* variants of a certified candidate; all are value preserving *)
polish[cand_] := DeleteDuplicates[Flatten[{cand, safeSimplify[cand],
     safeSimplify[Quiet[Check[RationalizeDenominator[Together[cand]], cand]]],
     denestInnerSurds[cand], safeSimplify[denestInnerSurds[cand]]}]];

(* ------------------------------------------------------------------ *)
(* the core: multiplier search                                        *)
(* ------------------------------------------------------------------ *)
```

```
316
317 DenestCore[problem_, OptionsPattern[]] :=
318   Block[{$verbose = OptionValue["Verbose"], $certifyTime = OptionValue["CertifyTime"],
319     $trials = 0, $maxTrials = OptionValue["MaxTrials"], $deadline = AbsoluteTime[] + OptionValue["
       TimeBudget"],
320     $visited = <||>, $cap = OptionValue["MultiplierCap"]},
321    Module[{radicand, outerpower, reductionpower, multiplierstack, multipliers, multiplier, terms,
       history,
322      attempt, solution, updatehistory, stackchange, addmultipliers, forced, i1, i2, i3, r},
323     If[! ExactAlgebraicQ[problem] || ! nestedRadicalQ[problem], Return[problem]];
324     If[TrueQ[CertifiedEqualQ[problem, 0]], Return[0]];
325     reductionpower = reductionExponent[problem];
326     If[! IntegerQ[reductionpower] || reductionpower < 2, Return[problem]];
327     outerpower = 1/reductionpower;
328     radicand = problem^reductionpower;
329     (* fast path for quadratic surds *)
330     If[MatchQ[problem, Power[_, Rational[1, 2]]],
331      r = denestSurdPower[First[problem]];
332      If[r =!= problem && cheaperQ[r, problem] && CertifiedEqualQ[r, problem],
333       log["quadratic surd fast path: ", problem, " -> ", r]; Return[r]]];
334     log["problem  ", problem];
335     log["outer root: ", reductionpower];
336     forced = OptionValue["Multipliers"];
337     If[ListQ[forced],
338      multipliers = {{False, {0, Infinity, Infinity}}, forced},
339      terms = DeleteCases[Replace[#, {Times[a___, Alternatives[_Rational, _Integer], b___] :> a*b,
340            Alternatives[_Rational, _Integer] -> 0, Complex[a_, b_] :> Sign[b]*I}] & /@
341         summands[Expand[RationalizeDenominator[radicand]]], 0];
342      terms = SortBy[terms, Quiet[N[Re[#], 20]] &];
343      multipliers = Prepend[DeleteCases[complementaryMultiplier /@ terms, 1], 1];
344      multipliers = {{True, {0, Infinity, Infinity}}, DeleteDuplicates[multipliers]}];
345     solution = problem;
346     history = {{0, Infinity, Infinity}};
347     multiplierstack = {multipliers};
348     r = Catch[
349       For[i1 = 1, i1 <= Length[multiplierstack], i1++,
350        If[ListQ[multiplierstack[[i1, 2]]] =!= True,
351         multiplierstack[[i1]] = multiplierstack[[i1, -2]] @@ (multiplierstack[[i1, -1]])];
352        multipliers = multiplierstack[[i1]];
353        stackchange = False;
354        If[multipliers[[1, 2]] =!= history[[1]],
355         log["multiplier count: ", Length[multipliers[[2]]], "  from: ", multipliers[[1, 2, {1, -1}]]],
356         log["multiplier count: ", Length[multipliers[[2]]], "  multipliers: ", multipliers[[2]]]];
357        For[i2 = 1, i2 <= Length[multipliers[[2]]], i2++,
358         multiplier = multipliers[[2, i2]];
359         updatehistory = False;
360         addmultipliers = False;
361         If[KeyExistsQ[$visited, multiplier],
362          log["skipping visited multiplier ", multiplier];
363          multiplierstack[[i1, 2, i2]] = {multiplier, $visited[multiplier]};
364          Continue[]];
365         If[$trials >= $maxTrials || AbsoluteTime[] > $deadline,
366          log["budget exhausted after ", $trials, " trials"];
367          Throw[Null, "budget"]];
368         $trials++;
369         attempt = If[And[Length[history] >= 3, history[[-2]] =!= history[[1]]],
370           tryMultiplier[problem, radicand, outerpower, multiplier, history[[-1, -1]], True],
371           tryMultiplier[problem, radicand, outerpower, multiplier, history[[-1, -1]], False]];
372         $visited[multiplier] = attempt[[2, -1]];
373         multiplierstack[[i1, 2, i2]] = {multiplier, attempt[[2, -1]]};
374         If[And[multipliers[[1, 1]] === True, attempt[[2, -1]] <= history[[-1, -1]]], addmultipliers =
       True];
375         If[attempt[[1]] === True,
376          updatehistory = True;
377          If[history[[-1]] =!= history[[1]], stackchange = True];
378          If[Length[attempt] == 3,
379           log["success: ", attempt[[2, {1, -1}]]];
380           solution = If[cheaperQ[attempt[[-1]], solution], attempt[[-1]], solution];
```

```
381           If[Or[history[[-1]] === history[[1]], history[[2, -1]]/attempt[[2, -1]] >= reductionpower],
382            history = Append[history, attempt[[2]]];
383            Throw[Null, "solution"],
384            log["attempting additional denesting, current solution is: ", solution]]];
385          If[And[history[[-1]] =!= history[[1]], Or[attempt[[2, -1]] < history[[-1, -1]], history[[-2]]
     === history[[1]]]],
386           log["improvement  ", attempt[[2, {1, -1}]]];
387           addmultipliers = True]];
388         If[addmultipliers === True,
389          Module[{smaller, larger, done, new},
390           If[attempt[[2, -1]] <= history[[-1, -1]],
391            smaller = attempt[[2]]; larger = history[[-1]],
392            smaller = history[[-1]]; larger = attempt[[2]]];
393           new = {{False, smaller}, newMultipliers, {smaller, If[larger =!= history[[1]], larger, Null],
     reductionpower}};
394           log["adding new multipliers from: ", multiplier];
395           If[multipliers[[1, 1]] === False,
396            done = {multipliers[[1]], multipliers[[2, 1 ;; i2]]};
397            multiplierstack = Insert[multiplierstack, new, i1 + 1];
398            If[i2 < Length[multipliers[[2]]],
399             multiplierstack = Insert[multiplierstack, {multipliers[[1]], multipliers[[2, i2 + 1 ;;
     -1]]}, i1 + 2]];
400            multiplierstack[[i1]] = done,
401            stackchange = False;
402            For[i3 = Length[multiplierstack], i3 >= i1, i3--,
403             If[Or[multiplierstack[[i3, 1, 2]] === history[[1]], new[[1, 2, -1]] > multiplierstack[[i3,
     1, 2, -1]],
404               And[new[[1, 2, -1]] === multiplierstack[[i3, 1, 2, -1]],
405                LeafCount[new[[1, 2, 1]]] >= LeafCount[multiplierstack[[i3, 1, 2, 1]]]]],
406              multiplierstack = Insert[multiplierstack, new, i3 + 1];
407              Break[]]]]];
408         If[updatehistory === True,
409          For[i3 = Length[history], i3 >= 1, i3--,
410           If[history[[i3, -1]] > attempt[[2, -1]],
411            history = Insert[history, attempt[[2]], i3 + 1];
412            Break[]]]];
413         If[stackchange === True, Break[]]];
414        (* products of promising initial multipliers, after the first pass *)
415        If[And[i1 === 1, ! ListQ[forced]],
416         Module[{worst, extra, ff, good},
417          good = Cases[multiplierstack[[i1, 2]], {_, _?NumericQ}];
418          If[Length[good] >= 2,
419           worst = Max[good[[All, 2]]];
420           extra = DeleteCases[DeleteDuplicates[
421               Replace[(Distribute[ff @@ (#^Range[0, 1] & /@ (Cases[good, {mult_, degree_} /; degree <
     worst][[All, 1]])), List] //. {
422                   ff[a___, Power[base_Times, exp_], b___] :> ff[a, b, Sequence @@ ((List @@ base)^exp)
     ],
423                   ff[a___, prod_Times, b___] :> ff[a, b, Sequence @@ prod],
424                   ff[a___, Power[base_, exp_], b___] :> ff[a, b, (-1)^exp, (-base)^exp] /; And[base !=
      -1, TrueQ[Re[base] < 0]],
425                   ff[a___, Power[base_, Rational[num1_, denom_]], b___, Power[base_, Rational[num2_,
     denom_]], c___] :>
426                    ff[a, b, c, base^(Mod[num1 + num2, denom]/denom)]}) /. ff -> Times,
427                Times[a___, _Integer, b___] :> a*b, {1}] //. Power[base_, Rational[a_, b_]] :> base^(
     Mod[a, b]/b)],
428              Alternatives @@ multiplierstack[[i1, 2, All, 1]]];
429           extra = Select[extra, ExactAlgebraicQ[#] && ! TrueQ[# == 0] &];
430           If[Length[extra] > $cap, extra = Take[complexitySort[extra], $cap]];
431           If[Length[extra] >= 1,
432            log["adding multipliers based on progress in initial guess"];
433            multiplierstack = Insert[multiplierstack, {multiplierstack[[i1, 1]], extra}, i1 + 1]]]]]],
434         "solution" | "budget"];
435       log["history: ", #[[{1, -1}]] & /@ history];
436       log["trials: ", $trials];
437       If[solution =!= problem && CertifiedEqualQ[solution, problem], solution, problem]]];
438
439   (* complementary multiplier for one term of the expanded radicand *)
```

```
complementaryMultiplier[term_] := Times @@ (complementaryFactor /@ factorsOf[term]);
complementaryFactor[Power[b_, Rational[n_, d_]]] := b^((d - n)/d);
complementaryFactor[Complex[0, _]] := -I;
complementaryFactor[e_] := Module[{deep = Cases[e, Power[_, _Rational], {0, Infinity}], maxd},
   If[deep === {}, 1,
    maxd = Max[RadicalDepth /@ deep];
    Times @@ (complementaryFactor /@ Select[deep, RadicalDepth[#] == maxd &])]];

(* one trial: {progress?, {multiplier, minpoly, degree}, [certified answer]} *)
tryMultiplier[problem_, radicand_, outerpower_, multiplier_, best_, worseknown_] :=
  Module[{arg1, r = 1/outerpower, minpoly, degree, polygcd, possibilities, roots, certified, result},
   result = {False, {multiplier, Null, Infinity}};
   If[! ExactAlgebraicQ[multiplier] || TrueQ[Quiet[PossibleZeroQ[multiplier, Method -> "ExactAlgebraics"
     ]]],
    log["invalid multiplier skipped: ", multiplier]; Return[result]];
   minpoly = Quiet[Check[MinimalPolynomial[(multiplier*radicand)^outerpower, arg1], $Failed]];
   If[! PolynomialQ[minpoly, arg1], log["minimal polynomial failed for ", multiplier]; Return[result]];
   degree = Exponent[minpoly, arg1];
   result = {False, {multiplier, Function[Evaluate[minpoly /. arg1 -> #]], degree}};
   log["multiplier: ", multiplier, "   minpoly degree: ", degree];
   If[Or[degree <= best, GCD[degree, best] < best],
    polygcd = Quiet[Check[PolynomialGCD[minpoly, arg1^r - multiplier*radicand, Extension -> Automatic],
     $Failed]];
    If[! PolynomialQ[polygcd, arg1], log["PolynomialGCD failed for ", multiplier]; Return[result]];
    If[And[0 < Exponent[polygcd, arg1] < r],
     roots = Quiet[Check[arg1 /. Solve[polygcd == 0, arg1], {}]];
     roots = Select[Flatten[{roots}], ExactAlgebraicQ];
     possibilities = Flatten[Outer[Times, roots/(multiplier^outerpower), Exp[2*I*Pi*Range[0, r - 1]/r
     ]]];
     certified = Select[possibilities, CertifiedEqualQ[#, problem] &];
     If[certified =!= {},
      certified = DeleteDuplicates[Flatten[polish /@ certified]];
      result = {True, result[[2]], pickCheapest[certified]},
      log["GCD of degree ", Exponent[polygcd, arg1], " but no certified candidate for ", multiplier];
      If[Or[degree < best, And[GCD[degree, best] < best, worseknown =!= True]], result[[1]] = True]],
     If[polygcd =!= 1 && Exponent[polygcd, arg1] > 0,
      If[Or[degree < best, And[GCD[degree, best] < best, worseknown =!= True]], result[[1]] = True],
      log["PolynomialGCD returned a constant: ", {radicand, outerpower, multiplier}]]],
    If[And[worseknown =!= True, degree != best],
     log["previous progress identified"];
     result[[1]] = True]];
   result];

(* generate a batch of multipliers from a discriminant, with a real cap *)
newMultipliers[smaller_, larger_, reductionpower_] :=
  Module[{arg1, new, primes, multiplierprimes, minprimes = 5, primeratio = 100, count, t, i3, disc},
   If[Or[larger === Null, GCD[smaller[[-1]], larger[[-1]]] == Min[smaller[[-1]], larger[[-1]]]],
    disc = Quiet[Check[Discriminant[smaller[[2]][arg1], arg1], $Failed]];
    If[! IntegerQ[disc] || disc == 0, Return[{{False, smaller}, {}}]];
    primes = DeleteCases[FactorInteger[disc, Automatic], {-1, 1}];
    primes = Select[primes, If[PrimeQ[#[[1]]], True, log["composite factor dropped: ", #]; False] &];
    primes = SortBy[primes, First][[All, 1]];
    multiplierprimes = primes;
    For[i3 = Length[primes], And[i3 > minprimes, multiplierprimes[[i3]]/multiplierprimes[[i3 - 1]] >
     primeratio], i3--,
     multiplierprimes[[i3]] = 0];
    multiplierprimes = DeleteCases[multiplierprimes, 0];
    (* largest t with reductionpower^t - 1 <= cap, computed in integer arithmetic *)
    t = 0; While[reductionpower^(t + 1) - 1 <= $cap, t++];
    count = Min[Max[1, t], Length[multiplierprimes]];
    multiplierprimes = Take[multiplierprimes, count];
    new = smaller[[1]]*Drop[Union[Divisors[(Times @@ multiplierprimes)^(reductionpower - 1)], primes],
     1],
    log["GCD of smallest degrees is smaller than any observed degree: ", {smaller[[{1, -1}]], larger
     [[{1, -1}]]}];
    new = {smaller[[1]]*larger[[1]]};
   new = DeleteCases[new, m_ /; KeyExistsQ[$visited, m]];
   {{False, smaller}, complexitySort[new]};
```

```
502
503 End[];
504 EndPackage[];
```

# C Experiment harness

All scripts are in `src/code-review/unified/harness/`; the recorded outputs are in `src/code-review/unified/lo`
The runner used for the original program is listed below; `runner_fixed.wl` is identical except that it loads the corrected package. The case files (`exp_original.wl`, `exp_helpers.wl`, `exp_round2.wl`, `exp_round3.wl`, `exp_fixed.wl`), the randomized families (`exp_fuzz.wl`, `exp_fuzz_fixed.wl`), the table generator (`export_tables.wl`), the script that exercises the reviews' proposed kernels (`test_review_kernels.wl`) and the shell script that runs the reviews' suites (`run_suites.sh`) are self-contained apart from absolute paths at their top.

```
(* Generic experiment runner for Strad.wl analysis *)
$srcFile = "C:/RadicalDenest/.claude/worktrees/strad-denesting-analysis-6158e6/src/original/Strad.wl";
$printAttrsBefore = Attributes[Print];
Get[$srcFile];
$printAttrsAfter = Attributes[Print];

SetAttributes[capture, HoldAll];
capture[expr_] := Module[{msgs = {}, res},
  res = Internal`HandlerBlock[{"Message",
      Function[m, If[TrueQ[m[[2]]], AppendTo[msgs, ToString[Extract[m, {1, 1}, HoldForm]]]]]}, expr];
  {res, DeleteDuplicates[msgs]}];

radDepth[e_] := If[AtomQ[e], 0,
  If[MatchQ[e, Power[_, _Rational]], 1 + radDepth[e[[1]]],
   Max[Prepend[radDepth /@ (List @@ e), 0]]]];

SetAttributes[runCase, HoldAll];
runCase[id_, expr_, expected_, tlimit_: 120] := Module[
  {t, res, msgs, out, numQ, numOK, exactOK, inputVal, depthIn, depthOut},
  inputVal = ReleaseHold[Hold[expr] /. {HoldPattern[Strad[e_, ___]] :> e, HoldPattern[DenestRadicals3[e_
     , ___]] :> e, HoldPattern[denest11[e_, ___]] :> e}];
  {t, {res, msgs}} = AbsoluteTiming[TimeConstrained[capture[expr], tlimit, {$TimedOut, {"TIMEOUT"}}]];
  numQ = NumericQ[inputVal] && NumericQ[res];
  numOK = If[numQ, Quiet[TrueQ[Abs[N[res - inputVal, 40]] < 10^-30]], "n/a"];
  exactOK = If[expected === None, "n/a",
     Quiet[TimeConstrained[TrueQ[RootReduce[res - expected] === 0], 30, "rr-timeout"]]];
  depthIn = radDepth[inputVal]; depthOut = radDepth[res];
  out = <|"id" -> id, "input" -> ToString[HoldForm[expr], InputForm],
    "result" -> ToString[res, InputForm], "time" -> Round[t, 0.001],
    "numeric" -> numOK, "exact" -> exactOK, "depth" -> {depthIn, depthOut},
    "messages" -> msgs|>;
  Print[id, " | ", ToString[HoldForm[expr], InputForm], "\n   -> ", ToString[res, InputForm],
    "\n   time ", Round[t, 0.001], "s  numeric=", numOK, "  exact=", exactOK,
    "  depth ", depthIn, "->", depthOut, If[msgs =!= {}, "  MSGS: " <> ToString[msgs], ""]];
  out];

Print["Kernel: ", $Version];
Print["Print attributes before load: ", $printAttrsBefore, "  after: ", $printAttrsAfter];
```

**Reproduction.** With Wolfram 15 and `wolframscript` on the path, from `src/code-review/unified/harness`:

```
wolframscript -file exp_original.wl      # Tables 6-10
wolframscript -file exp_helpers.wl       # Tables 11-12
wolframscript -file exp_round2.wl        # Tables 13-14
wolframscript -file exp_round3.wl        # Table 15
wolframscript -file exp_fuzz.wl          # Table 16, original
wolframscript -file exp_fixed.wl         # Tables 18-22
wolframscript -file exp_fuzz_fixed.wl    # Table 16, corrected
wolframscript -file test_review_kernels.wl
./run_suites.sh                          # the reviews' own suites
```

```
wolframscript -file export_tables.wl     # regenerates tables/*.tex
```

Only one kernel should run at a time on a single-seat license. Loading the original file in a valuable interactive session is not advisable: it clears several global symbols and unprotects `Print`.